Dossier de projet

RNCP niveau 6

Concepteur développeur d'applications

Trazyn's trials

SOUQUET Grégory

Holberton School Thonon les bains

24/03/2026

Table des matières

Dossier de projet....1
RNCP niveau 6....1
Remerciements....4
Résumé....5
Introduction....6
Compétences du référentiel....8
Besoins du projet....9
1. Contexte....9
2. Cahier des charges....10
a)Besoins initiaux....10
b)Besoins fonctionnels....10
3. Contraintes et livrables attendus 13
Gestion du projet....15
1. Environnement humain et technique....15
2. Objectifs qualité....17
Réalisations front-end....18
1. Organisation du code....19
a)Composants....19
b)Hooks et logique API....22
c)Modèles (types TypeScript)....23
d)Routing....24
e)Guards (protection des routes)....24
f)Rendu conditionnel et listes (équivalent directives / pipes)....25
2. Organisation de l'interface....26
a)Maquettes....26
b)Adaptation selon la largeur d'écran (non "mobile-first")....26
c)Plan du site (sitemap)....27
3. Interface utilisateur – présentation générale et exemples....28
a)Page de connexion....28
b)Interface de jeu – Aspect général....29
c)Interface de jeu – le plateau de jeu....30
d)Interface de jeu – Unités....33
e)Interface de jeu – le panneau de contrôle....36
Réalisation Back-end....41
1. Structure du code....42
2. Base de données....44
3. API RESTful....45
4. Composants métier....46
Eléments de sécurité....48
1. Authentification et autorisation....48

2. Protection contre les failles XSS....49
3. Validation des données et injection....49
4. Stockage sécurisé des informations sensibles....50
Jeux d'essai....51
Exemple : Analyzer.py + vérification visuelle....51
Respect du RGPD....53
Déploiement....54
Veille durant le projet....55
Déploiement Docker sur NAS Synology....56
Conclusion....57
Annexe 1 : Liste des librairies....59
Annexe 2 : Composants....60
Annexe 3 : Plan du site et Roadmap....61
Annexe 4 : Decision Tree 62
Annexe 5 : Script BDD auth....63
Annexe 6 : Tableau des routes API 64
Annexe 7 : Extraits de code....65
Annexe 8 : step.log....67
Annexe 9 : output de Analyzer.py....70

Remerciements

Je souhaite tout d'abord exprimer ma gratitude à l'équipe pédagogique de Holberton School, pour leur enseignement de qualité, leur disponibilité et leurs conseils avisés tout au long de cette formation.

Je remercie particulièrement Sarah Benamor, pour son accompagnement personnalisé, son écoute attentive, et ses retours constructifs, qui m'ont permis de progresser et de mener ce projet à bien, ainsi que Kevin Viais et Haris Hammache pour leur disponibilité, leur bienveillance et leurs retours techniques.

Je remercie ma famille et mes amis, pour leur soutien et leur implication sans lesquels je n'aurais pas pu mener ce projet jusqu'à son niveau actuel.

Enfin, je remercie le jury de soutenance pour l'attention qu'il portera à ce projet et pour ses questions qui me permettront, je l'espère, de progresser encore.

Résumé

Le projet qui vous est présenté dans ce mémoire est une application de simulation tactique inspirée de Warhammer 40'000, le wargame sur table. Le projet combine un moteur de jeu par phases, une API Flask, une interface React/TypeScript et un pipeline d'entrainement par apprentissage par renforcement.

L'architecture est organisee en modules (`engine/`, `ai/`, `services/`, `frontend/`, `config/`) afin de separer clairement les responsabilites.

La documentation technique et les outils de vérification de la conformite du code (aussi bien au niveau du respect des règles du jeu que des règles de codage) ont été mis en place tout au long du processus de developpement.

Le coeur IA s'appuie sur Stable-Baselines3 et sb3-contrib (MaskablePPO) pour gerer des actions contraintes.

Introduction

La conception et le développement de de Trazyn's Trials a tout d'abord été réalisé dans le cadre de l'obtention de mon diplôme de spécialisation en Machine Learning.

En effet, son développement a été initié dans le cadre du projet de fin d'année de la formation Machine Learning que propose Holberton School.

Ayant apprécié la partie Reinforcement Learning, j'ai décidé de pousser le curseur un peu plus loin que ce que j'avais appris : non seulement l'IA jouerait à un jeu, mais en plus, l'utilisateur pourra jouer contre elle.

J'ai ensuite voulu joindre l'utile à l'agréable en basant ce jeu sur l'univers de Warhammer 40'000, sans avoir pleinement conscience de la quantité astronomique de contraintes et de règles de jeu que cela implique. Pourtant, les 60 pages de règles exposant les bases du jeu et toutes les règles spécifiques aux centaines d'unités réparties dans les dizaines de factions différentes auraient dû me mettre la puce à l'oreille.

Bien qu'ayant une bonne expérience en scripts shells et ayant codé en Python pendant tout mon cursus Holberton, un aspect de ce projet m'était étranger : l'interface du jeu.

Limité par le temps disponible avant mon examen final, j'ai décidé de faire de cette difficulté une opportunité : Non seulement j'allais découvrir un autre aspect technique, mais en plus, je le ferai dans les temps en travaillant avec une IA.

Ainsi, tout au long du développement de ce jeu, j'ai non seulement appris à structurer et développer un jeu video complet, mais en plus, j'ai appris à coder avec une IA, avec toutes les joies et les peines que cela a pu entrainer.

Par exemple, le moment le plus difficile dans la mise en place du jeu a été le moment du cablage du moteur Python du backen avec le frontend Typescript.
En effet, d'un coté je m'assurais que mes agents suivent bien les règles pendant l'entrainement, et de l'autre, je vérifiais visuellement comment se déroulait une partie PvP.

Or, quand il a fallu jouer contre un agent, ce fut le drame : le backend et le frontend se faisaient concurrence dans la gestion des mécaniques de jeu.
J'ai donc du reprendre le frontend depuis le début en m'assurant que le backend soit la seule source de vérité.

Warhammer 40,000 se joue en une série de rounds de bataille. Lors de chaque round, chaque joueur joue un tour. Le même joueur prend toujours le premier tour à chaque round – la mission jouée stipulera de quel joueur il s’agit. Chaque tour consiste en une série de phases

(commandement, mouvement, tir, charge, combat), qu’il faut résoudre dans l'ordre.

Le jeu reproduit cette structure (round, tour, phase) tout en proposant des agents IA capables de prendre des décisions valides et performantes. Le projet associe un moteur de jeu Python (gymnasium), une API Flask, une interface React/TypeScript (PIXI pour le plateau) et un pipeline d’apprentissage par renforcement (Stable-Baselines3, MaskablePPO).

Objectifs du projet :

1. Concevoir un moteur de jeu robuste et conforme aux regles metier ;
2. Exposer ce moteur via une API backend exploitable par une interface web ;
3. Proposer une interface de jeu et de replay ;
4. Entrainer et evaluer des agents RL sur des scenarios parametrables ;
5. Documenter l’ensemble selon les attendus RE/REAC CDA.

Compétences du référentiel

Liste des compétences du référentiel				
N° Fiche AT	**Activités types**	**N° Fiche CP**	**Compétences professionnelles**	**Référence dans le mémoire (section, partie)**
1	Développer une application sécurisée	1	Installer et configurer son environnement de travail en fonction du projet	Gestion du projet → 1. Environnement humain et technique → Environnement technique (p. 10–11) : stack (Python, Flask, SQLite, React, TypeScript, Vite, PIXI, Git), Contrôle de version, Déploiement (Docker Compose).
1	Développer une application sécurisée	2	Développer des interfaces utilisateur	Réalisations front-end (p. 18–27) : 1. Organisation du code ; 2. Organisation de l'interface et maquettes ; 3. Interface utilisateur – écrans et exemples (SPA React, plateau PIXI, log, parcours auth → jeu → replay). Plan du site : p. 27.
1	Développer une application sécurisée	3	Développer des composants métier	Réalisation back-end (p. 38–42) : 1. Structure du code (W40KEngine, phase_handlers, observation_builder, action_decoder, reward_calculator) ; 4. Composants métier. Réalisations IA (p. 44–45) : pipeline d'entraînement, observation, récompenses.
1	Développer une application sécurisée	4	Contribuer à la gestion d'un projet informatique	Gestion du projet (p. 10–11, 16) : 1. Environnement humain et technique (démarche itérative, planning, état d'avancement Roadmap.md) ; 2. Objectifs de qualité (p. 16). Besoins du projet → 3. Contraintes et livrables (p. 9 et suiv.).
2	Concevoir et développer une application sécurisée organisée en couches	5	Analyser les besoins et maquetter une application	Besoins du projet (p. 9 et suiv.) : 2. Cahier des charges (besoins initiaux, fonctionnels, non fonctionnels). Réalisations front-end → 2. Organisation de l'interface et maquettes (p. 18–27, parcours, plan du site / sitemap, maquettes).
2	Concevoir et développer une application sécurisée organisée en couches	6	Définir l'architecture logicielle d'une application	Résumé / Introduction (p. 6–7) : architecture en modules (engine/, ai/, services/, frontend/, config/). Réalisation back-end (p. 38–39) : Architecture logicielle (Moteur, IA, API, Frontend, Config) ; 1. Structure du code.
2	Concevoir et développer une application sécurisée organisée en couches	7	Concevoir et mettre en place une base de données relationnelle	Réalisation back-end → 2. Base de données (p. 42) : SQLite config/users.db, tables (profiles, users, game_modes, options, profile_game_modes, profile_options, sessions), script initialize_auth_db, spécification USER_ACCESS_CONTROL.md. Annexes : schéma MEA / physique, script BDD auth.
2	Concevoir et développer une application sécurisée organisée en couches	8	Développer des composants d'accès aux données SQL et NoSQL	Réalisation back-end → 2. Base de données (p. 42, accès auth, requêtes paramétrées) ; 3. API RESTful (services/api_server.py, routes auth/game/replay). Pas de NoSQL dans le projet ; accès données = SQLite + API.
3	Préparer le déploiement d'une application sécurisée	9	Préparer et exécuter les plans de tests d'une application	Gestion du projet → 2. Objectifs qualité (p. 16, traçabilité, conformité). Jeux d'essai (p. 49) : plan de tests (test moteur, check_ai_rules, audit_shooting_phase, analyzer, hidden_action_finder, bot_evaluation, métriques). Réalisations IA → Conformité et analyse des logs. Jeu d'essai représentatif : Analyzer.py + vérification visuelle (Jeu_essai_complet.md).
3	Préparer le déploiement d'une application sécurisée	10	Préparer et documenter le déploiement d'une application	Déploiement (p. 51–52) : procédure sur NAS Synology, Docker Compose, configuration (ports, reverse proxy, volumes, variables d'environnement), schéma de déploiement. Documentation : Deployment_Synology.md.
3	Préparer le déploiement d'une application sécurisée	11	Contribuer à la mise en production dans une démarche DevOps	Déploiement (p. 51–52) : Docker Compose (backend + frontend Nginx), déploiement sur NAS Synology, healthcheck (/api/health), variables d'environnement (SYNO_CONFIG_PATH, SYNO_MODELS_PATH, SYNO_RUNTIME_PATH), flux applicatif. Besoins : application conteneurisable et déployable.

Besoins du projet

1. Contexte

La génèse
Le projet est issu d'un constat : Warhammer 40'000 est un jeu passionnant, mais on n'a pas toujours des amis disponibles pour jouer à un wargame dont l'essence est de se jouer à deux minimum.
C'est pourquoi je me suis lancé dans la conception d'un simulateur tactique basé sur le jeu Warhammer 40K dans sa forme la plus pure : le jeu sur table. Mon simulateur devrait respecter toutes les règles de Warhammer à la lettre, et proposer des agents IA entraînés par renforcement. Les besoins initiaux étaient : disposer d'un moteur fiable et conforme à des règles métier explicites (round, tour, phases, activation séquentielle, Lignes de vue, etc..), d'une API pour piloter des parties depuis une interface web, et d'un pipeline d'entraînement reproductible pour faire évoluer les agents.

Contexte de réalisation
Le projet Trazyn's Trials n'a pas été réalisé en entreprise mais dans le cadre de la formation Mastère Machine Learning & IA dispensée par Holberton School (Thonon-les-Bains). La réalisation a été effectuée en autonomie, avec un encadrement pédagogique et des points d'étape organisés par l'école.

Structure d'accueil
Holberton School est un établissement de formation aux métiers du développement logiciel, proposant des parcours en développement web et en intelligence artificielle. Le projet s'inscrit dans la continuité du diplôme de spécialisation Machine Learning (même école), dont le projet de fin d'année a servi de point de départ au moteur de jeu et au pipeline d'apprentissage par renforcement. Le passage au cursus CDA a permis d'ajouter et de formaliser les dimensions « application web » : API, frontend, authentification, déploiement, conformité au référentiel CDA.

Environnement humain
Le développement a été mené en **solo** : pas d'équipe projet ni de chef de projet interne. Les échanges avec l'équipe pédagogique (Sarah Benamor, Kevin Viais, Haris Hammache) ont porté sur le suivi de l'avancement, les retours méthodologiques et techniques, et la conformité au cahier des charges de la formation. Aucune réunion de projet formelle ni compte rendu de réunion n'ont été produits au sens strict. Le suivi s'est fait via les outils de la formation et les livrables intermédiaires (code, documentation, présentations).

2. Cahier des charges

a) Besoins initiaux

Avant la création de Trazyn's Trials, pour jouer à Warhammer 40'000 les joueurs devaient :

- **Choisir son armée** : Avant de jouer, tout joueur doit commencer par choisir l'armée qu'il voudra jouer parmi les plus de 20 factions disponibles selon ses goûts esthetiques et son style de jeu.
- **Constituer son armée** : Une fois la faction choisie, le joueur doit acheter / peindre des dizaines de figurines afin de construire son armée.
- **Trouver un partenaire de jeu** : les cercles de jeu sont parfois rares et fermés. Le joueur doit faire un effort de recherche afin de dénicher un groupe qui l'acceptera, on convaincre un ami de se lancer dans l'aventure avec lui.
- **Plannifier la partie** : Une fois l'adversaire trouvé, il faut encore convenir avec lui d'un lieu, d'une date et d'un format de jeu qui convienne à chacun.
- **Préparer la table de jeu** : Une fois réunis, et avant de lancer la partie, il faut préparer la table et installer les décors.
- **Vérifier la conformité aux règles** : Pendant la partie, il faut très régulièrement (surtout au début) se référer aux règles afin de s'assurer qu'on les suive bien : il faut trouver quel livre / document contient l'information, trouver l'information et se mettre d'accord sur la règle est sujette à interprétation.

b) Besoins fonctionnels

Pour les joueurs :

Trazyn's Trials a pour but de palier à une grande partie de ces contraintes :

- **Le jeu :**
  - Proposer une variété de factions permettant à chacun de trouver celle qui lui correspond.
  - Pour chaque faction, proposer une quantité d'unité satisfaisante pour couvrir tous les aspects du jeu.
  - Proposer un adversaire IA stimulant.

  - Proposer des configurations de décors et d'objectifs variés.
  - Garantir que le moteur soit 100% conforme aux règles du jeu et guider le joueur dans son apprentissage.
- **Compte utilisateur :**
  - Création de compte.
  - Connexion et et déconnexion au compte.
  - Consultation du profile.
  - Historique des parties.

Pour les administrateurs :

- **Le jeu :**
  - La modularité du jeu doit faciliter l'ajout et le controle des factions.
  - Les parties ayant lieu sur des formats pré-définis, il faut proposer des gameplays assez variés et stimulants.
  - Proposer des configurations de décors et d'objectifs modulaires et variées.
  - Cadrer les actions et donner les informations au joueur pour qu'il comprenne.
- **L'opposition :**
  - Proposer un adversaire IA stimulant sans être frustrant.
  - Les factions doivent être équilibrées entre elles.
- **Les utilisateurs :**
  - Consulter la liste des utilisateurs.
  - Modifier le rôle d'un utilisateur.
  - Supprimer un utilisateur.

c. Besoins non fonctionnels

- **Fiabilité du jeu:**
  - Activation des unités et Transitions de phases déterministes.

  - Pas action invalide, pas d'activation inappropriée.
- **Maintenabilité** :
  - Architecture modulaire.
  - Documentation indexée.
  - Règles de codage.
  - Scripts de vérification.
- **Sécurité** :
  - Protection des données utilisateurs (hachage des mots de passe, token de session).
  - Restriction de l'accès aux fonctionnalités d'administration aux personnes désignées.
- **Portabilité**
  - Application conteneurisable et déployable sur NAS.

3. Contraintes et livrables attendus

Le développement de Trazyn's Trials s'est inscrit dans le cadre d'une formation intensive en Machine Learning, imposant un calendrier serré entre le début du projet et sa soutenance finale (huit semaines).

Aussi, il fut dans sa première phase très orienté IA, la gestion des utilisateurs et de l'exposition à internet n'étant pas évalués.

Ainsi, à l'issue de ma soutenance, je disposais d'un jeu qui fonctionnait au niveau backend, frontend, et ces deux éléments étaient bien coordonnés.

Cependant, ce n'était qu'une demo très simple, toutes les phases du jeu n'étaient pas implémentées.

C'est alors que j'ai du enchaîner avec le programme RNCP 6.

Le RNCP dédié à l'IA étant indisponible, j'ai dû me concentrer sur le RNCP développeur web.

Alors, alors que je travaillais toujours sur la complétion des mécaniques du jeu, j'ai du mettre l'accent sur la partie développement web.

Le projet devait privilégier une architecture modulaire (moteur, API, frontend, IA) et des technologies open source, avec un hébergement maîtrisé (Docker Compose, déploiement sur NAS Synology ou équivalent).

Sur le plan technique, le projet devait être compatible avec les navigateurs web modernes (Chrome, Firefox, Safari, Edge). L’interface est conçue pour un usage optimal sur ordinateur ; l’adaptation aux différentes tailles d’écran (tablette, smartphone) n’a pas été retenue comme objectif prioritaire, le plateau de jeu (canvas PIXI) restant dimensionné pour le bureau pour des raisons de lisibilité.

Le respect des contraintes légales, et en particulier du Règlement général sur la protection des données (RGPD), était un impératif. Le projet devait intégrer les principes de protection des données personnelles dès la conception (privacy by design) et garantir la sécurité des informations collectées (authentification, profils, sessions).

Les livrables attendus à l’issue du projet étaient les suivants :

- Une application web fonctionnelle (simulateur tactique, interface de jeu et de replay, modes PvP, PvE, Test et Debug selon les permissions), accessible en ligne via une URL publique (déploiement Docker, reverse proxy, par exemple sur NAS Synology).
- Le code source complet et commenté de l’application (frontend React/TypeScript, backend Flask/Python, moteur de jeu, pipeline IA), hébergé sur un dépôt Git (GitHub),

avec les configurations d'agents (training, rewards, scénarios) et les modèles entraînés.
- La documentation technique et les guides du projet, ainsi que les outils de vérification de conformité (scripts, analyzer).
- Le présent dossier de présentation du projet, détaillant les différentes phases de conception et de réalisation.
- Une présentation orale du projet, incluant une démonstration de l'application.

Gestion du projet

1. Environnement humain et technique

Le développement de Trazyn's Trials a été initié dans le cadre de le formation Machine Learning de Holberton School. Etant seul dans ma promotion, et devant composer avec des contraintes personnelles fortes, j'ai mené toute la formation et ce projet de façon autonome.

Le projet a été mené en suivant plusieurs étapes :

Démarche itérative et planning :

1. Moteur Python et handlers de phases de jeu.
2. API backend (Flask, routes jeu et authentification).
3. Entrainement IA (PPO, bots de référence, metriques).
4. Développement du frontend (React, plateau hex, log).
5. Intégration frontend aux contraintes backend (appels API, synchronisation d'état).
6. Déploiement et qualité (conformité règles, analyzer, check_ai_rules).

Environnement technique :

- **Backend** : L'API REST a été construite avec Python 3 et le framework Flask. La mécanique de jeu est en Python pur (moteur engine/, handlers de phases), sans dépendance serveur spécifique. La légèreté de Flask et son adéquation aux API REST en ont fait un choix adapté pour exposer le moteur et gérer l'authentification.
- **Base de données** : La persistance des données d'authentification (utilisateurs, profils, sessions) est assurée par SQLite (config/users.db), via le module standard Python. Sa simplicité et l'absence de serveur dédié conviennent à un projet de démonstration et à un déploiement conteneurisé.
- **Authentification** : La sécurisation de l'accès aux ressources sensibles (partie, replay, modes Debug/Test) repose sur un token bearer de session, vérifié sur chaque requête protégée. Les droits sont gérés par profil (base / admin), selon USER_ACCESS_CONTROL.md.
- **Hachage des mots de passe** : Le hachage est réalisé en PBKDF2-HMAC-SHA256 (200 000 itérations), afin de garantir une sécurité renforcée des données utilisateur.

- **IA** : L'entraînement des agents s'appuie sur Stable-Baselines3 (SB3), sb3-contrib (MaskablePPO) et gymnasium. MaskablePPO permet de restreindre les actions aux seules actions valides (masque binaire), en cohérence avec les règles métier du jeu.
- **Frontend** : L'interface est développée avec React 19, TypeScript 5.8, Vite 7, React Router, PIXI (rendu du plateau hex) et Tailwind CSS. React et TypeScript offrent une base maintenable ; PIXI assure un rendu fluide du plateau et des unités.
- **Gestionnaire de paquets** : pip et requirements.txt (ou requirements.runtime.txt pour le déploiement) sont utilisés pour les dépendances Python ; npm/pnpm pour le frontend.
- **Contrôle de version** : Git, avec hébergement sur GitHub, permet de suivre l'évolution du code, de conserver l'historique et d'assurer la sauvegarde du projet.
- **Déploiement** : L'application est conteneurisée avec Docker Compose (backend Flask, frontend servi par Nginx) et déployée sur un NAS Synology (DSM), avec reverse proxy HTTPS et volumes persistants (base auth, modèles IA, logs). Ce choix permet un hébergement maîtrisé et une démonstration en conditions réelles.
- **IDE** : Le développement a été réalisé avec l'éditeur Cursor, choisi pour son intégration avec l'assistance agentique et les outils de développement web.
- **Autres outils** : Scripts de conformité (scripts/check_ai_rules.py), analyse des logs d'entraînement (ai/analyzer.py, ai/hidden_action_finder.py), TensorBoard pour les métriques, éventuellement Postman ou équivalent pour tester l'API.
- **Librairies tierces** : Outre les briques déjà citées (Flask, React, SB3, PIXI, etc.), le projet utilise des librairies tierces pour la validation des données (require_key, require_present), le décodage des actions, les récompenses, etc. Une liste détaillée des librairies, avec versions et rôles, figure en **Annexe 1**.

2. Objectifs qualité

La qualité du produit final a été une préoccupation constante tout au long du projet. Les objectifs suivants ont été fixés :

- **Accessibilité** : Des media queries CSS (App.css) adaptent la mise en page (sidebar, log, tableaux) aux écrans de largeur réduite. L'application est conçue pour une utilisation optimale sur ordinateur ; l'usage sur tablette ou smartphone, à la lumière des contraintes fonctionnelles du jeu, n'a pas été pleinement validé.
- **Sécurité utilisateurs** : La protection des données utilisateur et la prévention des attaques ont été traitées avec attention. Les mots de passe sont hachés avec PBKDF2-HMAC-SHA256 (200 000 itérations) avant stockage en base. L'authentification repose sur un token bearer de session ; les routes sensibles (jeu, replay, modes Debug/Test) sont protégées côté serveur par vérification du token et du profil.
- **Sécurité applicative** : Les injections XSS sont limitées par des réponses JSON côté API et par l'échappement par défaut de React côté client. La validation stricte des données (require_key, require_present) et l'absence de fallback masquant les erreurs réduisent les risques d'injection ou de comportements indéterminés.
- **Utilisabilité** : Une attention particulière a été portée à l'ergonomie de l'application. L'interface (authentification, plateau hex, log des actions, replay) est pensée pour une navigation fluide. Les informations de partie (état, phase, unités) sont présentées de manière lisible. La cohérence visuelle est assurée par des composants réutilisables et par un parcours utilisateur aligné sur les modes de jeu (PvP, PvE, Replay).
- **Performance** : L'application a été conçue pour rester réactive. Le moteur de jeu (Python, single source of truth, pas de copie superflue du game_state) et l'API Flask restent légers ; le frontend (React, Vite) assure un chargement et des mises à jour sélectives et rapides. Le masquage des actions invalides (MaskablePPO) évite les calculs inutiles côté IA et accélère l'entraînement. La réactivité perçue est maintenue par des appels API ciblés et une mise à jour de l'état après chaque action.

Réalisations front-end

L’interface utilisateur de l’application a été développée avec React (version 19), TypeScript (5.8), Vite (7), React Router et PIXI pour le rendu du plateau hex, avec Tailwind CSS pour les styles.

Ce choix repose sur la richesse de l’écosystème React, une structure en composants et hooks favorable à la maintenabilité et à l’évolutivité, et sur TypeScript (typage statique, meilleure détection des erreurs).

Vite assure un build et un rechargement rapides en développement. PIXI permet un rendu graphique performant du plateau hexagonal et des unités, adapté à une simulation tactique.

L’ensemble s’inscrit dans une volonté d’utiliser des technologies modernes et adaptées à une application web complexe, amenée à évoluer (nouveaux modes, replay, intégration avec le moteur et l’API).

Cette section a pour objectif de présenter les réalisations frontend en mettant en évidence les compétences en développement React/TypeScript.

On décrira d’abord l’organisation du code source : composants, hooks, contexte ou store, routage et protection des routes selon les permissions.

On abordera ensuite la conception de l’interface : parcours utilisateur, maquettes fonctionnelles et techniques d’adaptation (media queries CSS pour les écrans réduits).

Enfin, on détaillera l’interface par profil (visiteur non connecté, utilisateur connecté, administrateur) en s’appuyant sur des captures d’écran annotées et des extraits de code pour illustrer les points clés (appels API, synchronisation avec le game_state, rendu PIXI, log des actions) et les bonnes pratiques mises en œuvre.

1. Organisation du code

Le frontend est structuré en composants (pages et composants réutilisables), hooks (ex. appel API vers /api/game/, gestion de l'état de partie) et contexte ou store pour l'état global (utilisateur connecté, permissions, état de jeu). Le routage (React Router) distingue les écrans authentification (/auth), jeu (/game avec paramètre de mode) et replay.

Les routes sensibles sont protégées en fonction du token et du profil (accès aux modes PvE, PvP, Replay). Les appels API sont centralisés (base URL relative /api pour limiter les problèmes CORS en production avec proxy Nginx).

a) Composants

Les composants sont les éléments de base de l'interface. Chaque composant regroupe logique et template dans un fichier .tsx (parfois avec un fichier .css ou des classes Tailwind). Il reçoit des données via des props typées et affiche du JSX.

Exemple : le composant GameLog

Il est responsable de l'affichage du log des actions. Il reçoit la liste des événements en entrée (events) et les affiche dans le template. La structure du composant (props, déstructuration, rendu) est la suivante :

```
interface GameLogProps {
  events: GameLogEvent[];
  maxEvents?: number;
  availableHeight?: number;
  useStepNumbers?: boolean;
  currentTurn?: number;
  debugMode?: boolean;
}

export const GameLog: React.FC<GameLogProps> = ({
  events,
  availableHeight = 220,
  useStepNumbers = false,
  debugMode = false,
}) => {
```

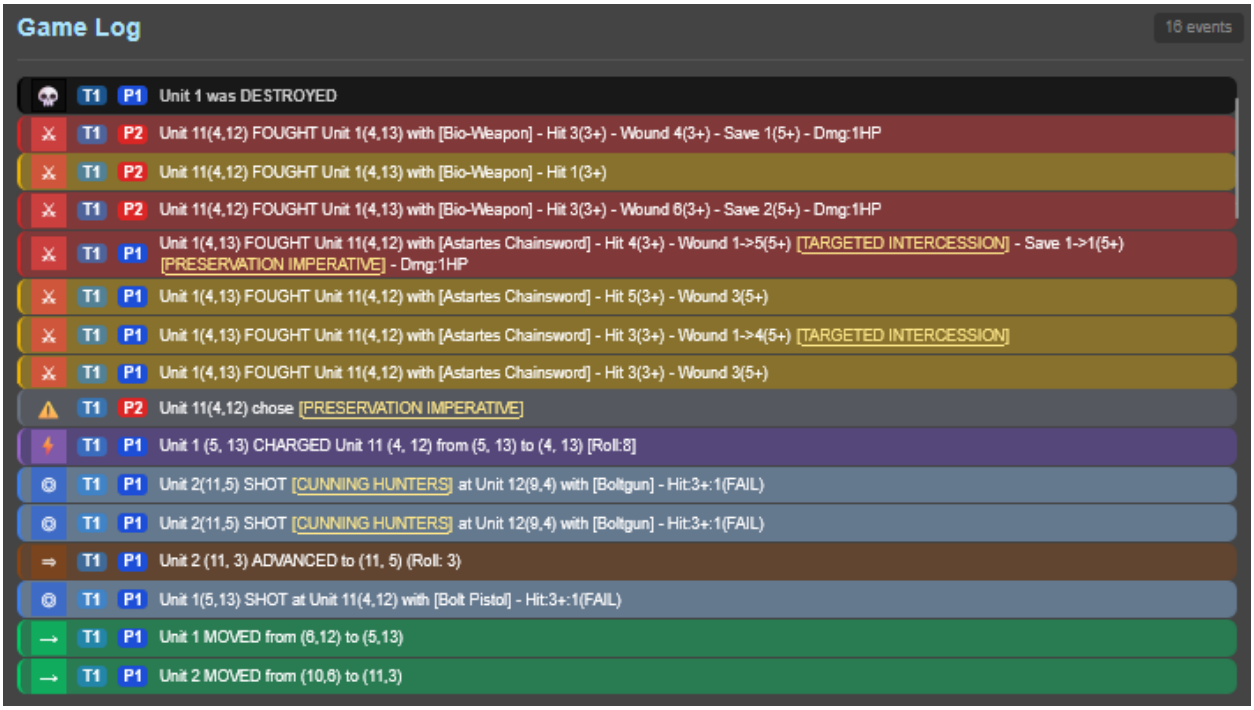
Interface GameLogProps et déclaration du composant GameLog : props typées (events, availableHeight, etc.) et déstructuration.

Le template JSX affiche soit un message lorsque la liste est vide, soit la liste des événements en les parcourant avec .map(). Chaque entrée utilise des classes CSS conditionnelles (type d'événement, action wait, contrôle d'objectif) :

```
        <div className="game-log__empty">No events yet. Start playing to see the action log!</div>
      ) : (
        <div
          ref={eventsContainerRef}
          className="game-log__events"
          style={{
            maxHeight: `${availableHeight}px`, // Use full available height
            overflow: "auto",
          }}
        >
          {displayedEvents.map((event) => {
            // Check if this is a wait/skip action
            // Multiple detection methods:
            // 1. Check action_name field
            const actionName = event.action_name || event.actionName;
            const hasWaitActionName =
              actionName &&
              (actionName.toLowerCase() === "wait" || actionName.toLowerCase() === "skip");

            // 2. Check event type (backend logs wait actions differently - check message instead)
            const isWaitType = false; // 'wait' is not a valid type in BaseLogEntry, check message instead

            // 3. Check message content (frontend logs "chose not to move")
            const message = event.message || "";
            const hasWaitMessage =
              message.toLowerCase().includes("chose not to move") ||
              message.toLowerCase().includes("chose not to charge") ||
              message.toLowerCase().endsWith(" wait");

            const isWaitAction = hasWaitActionName || isWaitType || hasWaitMessage;
            const waitClass = isWaitAction ? "game-log-entry--wait" : "";
            const isObjectiveControl = actionName === "objective_control";
            const objectiveControlClass = isObjectiveControl
              ? event.player === 1
                ? "game-log-entry--objective-control-p1"
                : event.player === 2
                  ? "game-log-entry--objective-control-p2"
                  : "game-log-entry--objective-control-neutral"
              : "";
            return (
              <div
                key={event.id}
                className={`game-log-entry ${getEventTypeClass(event)} ${waitClass} ${objectiveControlClass}`}
              >
```

Rendu conditionnel (liste vide ou liste d'événements) et boucle displayedEvents.map() avec classes conditionnelles (getEventTypeClass, waitClass, objectiveControlClass).

Pour illustrer le rendu à l'écran :



Vue du composant GameLog en situation de partie : log des actions avec événements triés et styles par type.

La liste complète des composants est disponible en **Annexe 2**.

b) Hooks et logique API

Les "services" côté React sont incarnés par des hooks et des modules qui encapsulent les appels API. Le hook useEngineAPI centralise la communication avec le backend (démarrage de partie, envoi d'action, tour IA, état). Les requêtes utilisent fetch avec une base d'URL commune et, pour les routes protégées, le token Bearer.

```
const API_BASE = "/api";
```

Constante API_BASE pour centraliser l'URL de l'API.

```
const authSession = getAuthSession();
if (!authSession?.token) {
  throw new Error("Session utilisateur manquante. Merci de vous reconnecter.");
}

const response = await fetch(`${API_BASE}/game/start`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${authSession.token}`,
  },
  body: JSON.stringify(requestPayload),
});
```

Récupération de la session (getAuthSession), vérification du token et requête POST /api/game/start avec en-tête Authorization Bearer.

```
const response = await fetch(`${API_BASE}/game/action`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: requestBody,
});

if (!response.ok) {
  throw new Error(`Action failed: ${response.status}`);
}

const data = await response.json();
```

Requête POST /api/game/action avec body JSON (action joueur) et gestion de la réponse

c) Modèles (types TypeScript)

Les types et interfaces définissent la structure des données (état de jeu, unité, phase, etc.). Ils permettent au compilateur TypeScript de vérifier les props, les retours d'API et l'état, et d'éviter les erreurs à l'exécution.

```
export type PlayerId = 1 | 2;
export type UnitId = number;
export type GamePhase = "deployment" | "command" | "move" | "shoot" | "charge" | "fight";
export type GameMode =
  | "select"
  | "movePreview"
  | "attackPreview"
  | "targetPreview"
  | "chargePreview"
  | "advancePreview";
// AI_TURN.md COMPLIANCE: Fight subphase names match backend exactly
export type FightSubPhase =
  | "charging"
  | "alternating_non_active"
  | "alternating_active"
  | "cleanup_non_active"
  | "cleanup_active"
  | null;
```

Types PlayerId, UnitId, GamePhase, GameMode, FightSubPhase et interface Position.

```
export type {
  ActionResult,
  AttackPreview,
  FightPhaseState,
  FightSubPhase,
  GameMode,
  GamePhase,
  GameState,
  MovePreview,
  PlayerId,
  Position,
  SemanticAction,
  ShootingPhaseState,
  SingleAttackState,
  SingleShotState,
  TargetPreview,
  Unit,
  UnitId,
} from "./game";
```

Réexport des types métier (GameState, Unit, GamePhase, etc.) pour un usage unifié dans l'application.

d) Routing

L'application utilise React Router. Les routes sont déclarées dans Routes.tsx (importé comme point d'entrée dans main.tsx) : /auth pour l'authentification, /game pour le jeu (avec protection par token et vérification des permissions de mode), /replay pour la visionneuse de replay, et / ou toute route inconnue redirigent selon l'état de connexion (RootRedirect).

```
export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/auth" element={<AuthPage />} />
        <Route path="/game" element={<ProtectedGameRoute />} />
        <Route path="/replay" element={<ProtectedReplayRoute />} />
        <Route path="/" element={<RootRedirect />} />
        <Route path="*" element={<RootRedirect />} />
      </Routes>
    </BrowserRouter>
  );
```

*Déclaration des routes dans Routes.tsx (point d'entrée réel de l'application) : /auth (AuthPage), /game (ProtectedGameRoute), /replay (ProtectedReplayRoute), / et * (RootRedirect vers /auth si non connecté, vers /game?mode=pve si connecté)*

e) Guards (protection des routes)

Il n'y a pas de guard Angular ; l'accès aux routes protégées est assuré côté API : le hook qui appelle le backend récupère la session (getAuthSession) et envoie le token. En l'absence de token, une erreur explicite est levée et l'utilisateur doit se reconnecter.

```
const authSession = getAuthSession();
if (!authSession?.token) {
  throw new Error("Session utilisateur manquante. Merci de vous reconnecter.");
}
```

Vérification de la session et du token avant l'appel à /api/game/start ; erreur explicite si session manquante.

f) Rendu conditionnel et listes (équivalent directives / pipes)

En React, l'affichage conditionnel s'exprime par des expressions && ou des ternaires ; la répétition d'éléments par .map() sur un tableau (équivalent de *ngFor). Les « pipes » sont remplacés par des fonctions (ex. formatage de date) ou du code inline.

```
<div className="game-log__content">
  {displayedEvents.length === 0 ? (
    <div className="game-log__empty">No events yet. Start playing to see the action log!</div>
  ) : (
    <div
      ref={eventsContainerRef}
      className="game-log__events"
      style={{
        maxHeight: `${availableHeight}px`, // Use full available height
        overflow: "auto",
      }}
    >
```

Affichage conditionnel : liste vide (message « No events yet ») ou conteneur scrollable pour la liste.

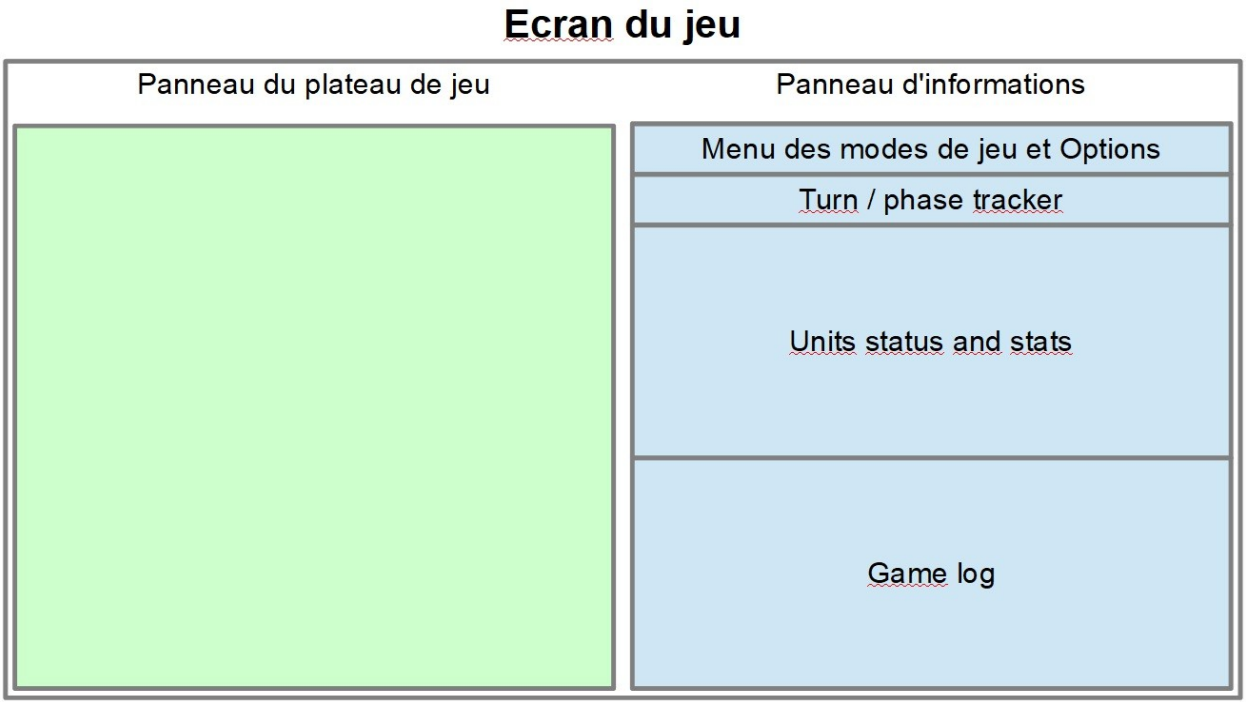
```
{displayedEvents.map((event) => {
  // Check if this is a wait/skip action
  // Multiple detection methods:
  // 1. Check action_name field
  const actionName = event.action_name || event.actionName;
```

Boucle sur displayedEvents avec .map(), key={event.id}, et classes CSS conditionnelles (équivalent ngFor et [ngClass]).

2. Organisation de l’interface

a) Maquettes

La conception de l’interface a été guidée par l’organisation des écrans (authentification, plateau de jeu, log, replay). Afin de fournir la meilleure expérience visuelle au joueur, toute la partie gauche de l'écran est réservée à l'affichage du plateau de jeu.
Tout le reste, le menu de navigation, les options, les informations sur le jeu et le log doivent tenir sur la partie droite de l'écran :



b) Adaptation selon la largeur d’écran (non “mobile-first”)

L’application n’est pas conçue comme responsive au sens “adaptée à tous les terminaux”. Pour des raisons d'ergonomie, elle vise en priorité un usage sur ordinateur.

Le plateau de jeu (canvas PIXI) n’a pas de logique de redimensionnement selon la taille de la fenêtre, ses dimensions sont fixes. Une utilisation confortable sur tablette ou smartphone n’a pas été visée ni validée.

Ce qui existe dans le code :
Dans App.css, des media queries (1400px, 900px, 768px, 480px) modifient uniquement la mise en page autour du plateau pour limiter les débordements quand la fenêtre est rétrécie :

- **≤ 1400px** : tableaux d'unités limités à 800px de large, police et padding réduits.
- **≤ 900px** : tableaux en pleine largeur, marge ajustée, défilement horizontal si besoin (overflow-x: scroll).
- **≤ 768px** : la zone principale passe en colonne (.app-container { flex-direction: column }) ; la barre latérale (sidebar) prend toute la largeur avec une hauteur max de 200px et une bordure en bas ; la zone de jeu (.game-area) s'empile aussi en colonne ; les tableaux d'unités passent en pleine largeur ; titre de la page en police plus petite.
- **≤ 480px** : padding du header et taille du titre encore réduits.

La barre de navigation (SharedLayout) reste toujours visible (boutons PvP, PvE, etc.). Le canvas PIXI du plateau ne s'adapte pas à la taille d'écran (pas de resize/scale selon viewport).

En résumé : L'interface est pensée pour le bureau. Les media queries servent à ce que la page ne "casse" pas sur des largeurs réduites (réorganisation en colonne, scroll), pas à offrir une expérience optimale sur mobile ou tablette.

c) Plan du site (sitemap)

- **Pages accessibles à tous** : /auth (connexion et inscription).
- **Pages réservées aux utilisateurs connectés** : /game (jeu, modes selon permissions), /replay (visionneuse de replay).
- **Options réservées au profil admin (ou permissions dédiées)** : modes Test et Debug (ex. ?mode=pve_test, ?mode=pvp_test). Sans droit, redirection vers un mode autorisé ou erreur.

Le plan du site et l'enchaînement des écran sont disponibles en **Annexe 3.**

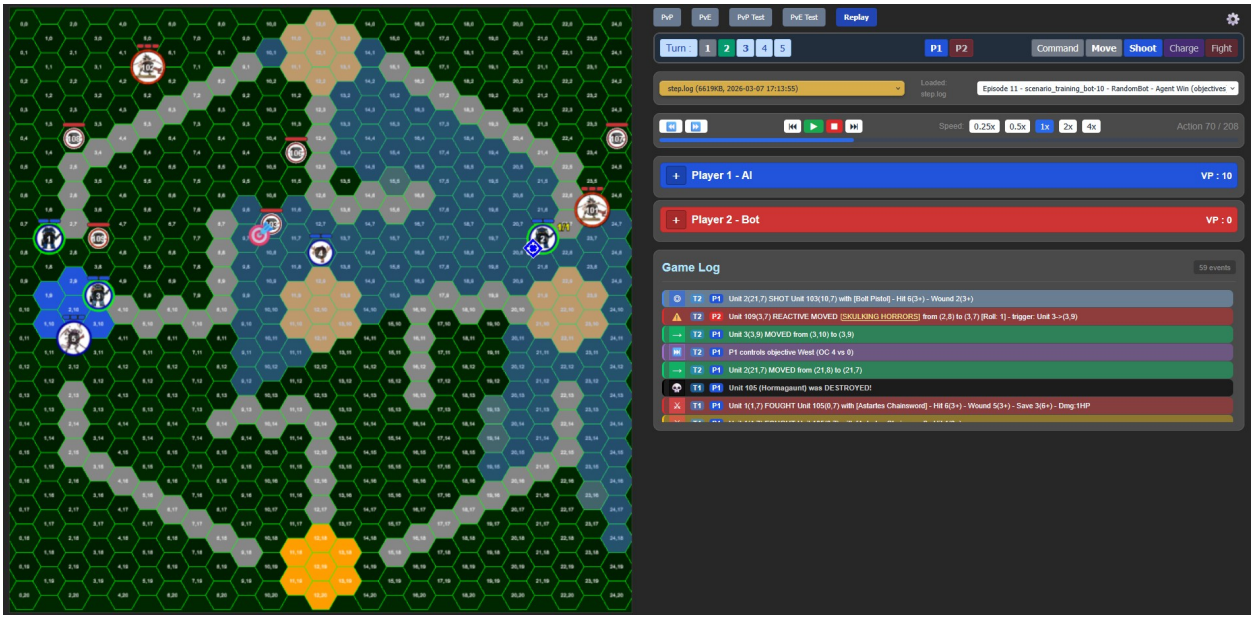
3. Interface utilisateur – présentation générale et exemples

a) Page de connexion

Le parcours de l'utilisateur dépend de l'état de connexion et du profil : un visiteur non connecté n'a accès qu'à la page d'authentification :

b) Interface de jeu – Aspect général

Les interfaces de jeu et de Replay sont quasiment identiques. L'interface de Replay dispose seulement de features supplémentaires pour la selection du replay et la navigation pendant le visionnage.



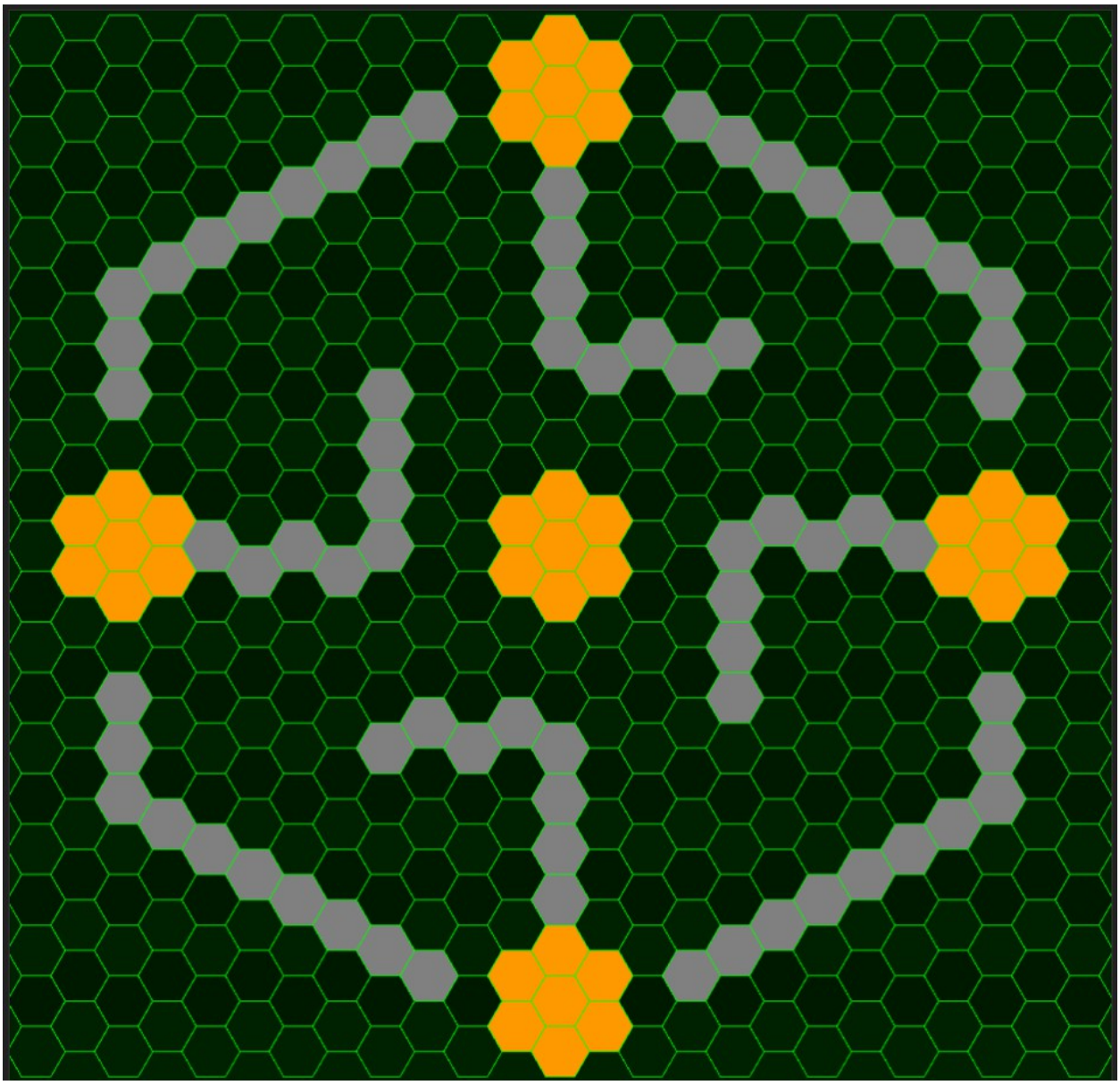
Partie gauche de l'interface : le plateau de jeu. C'est dans cette partie qu'est représentée la table de jeu, et ou se déroulent les actions.
On y voit touts les cases ainsi que les murs (en gris) et les objectifs (en jaune quand ils sont neutres, et rouge ou bleu selon le joueur les controlant).
Chaque unité est représentée par une icone, dont la taille varie selon le gabarit de la figurine dans le jeu sur table.

Partie droite de l'interface : le panneau de controle. Ici apparaissent les caractéristiques des unités et de leurs armes, ainsi que toutes les informations necessaires pour suivre le cours du jeu.

c) Interface de jeu – le plateau de jeu

La partie gauche de l'interface est consacrée à la visualisation de l'action sur le plateau de jeu.

Le plateau a été divisé en hexagones, ce qui est le meilleure compromis entre liberté de mouvement et économie de calculs pour l'entrainement de l'IA.



Murs

Les murs sont des hexagones impassables définis dans des fichiers JSON sous la forme d'une liste de coordonnées [col, row]. Chaque scenario peut appeler une configuration de murs différente sans avoir à les redéfinir localement.

Au chargement, ces hexagones sont normalisés et stockés dans game_state["wall_hexes"] (ensemble de tuples (col, row)).

Mouvement

- Un hex contenant un mur n'est jamais traversable : il est exclu des destinations valides et du parcours.
- Le calcul des déplacements utilise un BFS (breadth-first search) qui évite les murs et les hex déjà occupés par une unité. La distance « réelle » entre deux cases est donc la distance de chemin (nombre de pas), pas la distance en ligne droite.
- Les unités avec la règle FLY peuvent ignorer les murs pendant le trajet (elles ne comptent pas les hex de murs dans le coût de déplacement), mais ne peuvent toujours pas atterrir sur un hex mur : la destination finale doit rester valide (ni mur, ni occupée, et respecter les règles d'adjacence avec les unités ennemies).

Tir (ligne de vue)

- La ligne de vue entre tireur et cible est une ligne d'hexagones (algorithme de tracé de segment en grille hexagonale).
- Si un seul hex de mur se trouve sur ce tracé (entre le premier et le dernier hex, exclus), la ligne de vue est bloquée : la cible n'est pas dans le pool des cibles valides.
- Les murs sont donc des obstacles opaques pour le tir : on ne peut pas tirer à travers.

En résumé : les murs bloquent le passage au mouvement (sauf traversée pour FLY) et bloquent la ligne de vue pour le tir.

```
{
  "wall_id": "walls-01",
  "wall_hexes": [
    [2, 5], [2, 6], [2, 7], [3, 4], [4, 4], [5, 3], [6, 3], [7, 2], [8, 2], [9, 1],
    [16, 2], [17, 2], [18, 3], [19, 3], [20, 4], [21, 4], [15, 1], [22, 5], [22, 6], [22, 7],
    [2, 14], [2, 15], [2, 13], [3, 15], [4, 16], [5, 16], [6, 17], [7, 17], [8, 18], [9, 18],
    [22, 13], [22, 14], [22, 15], [21, 15], [20, 16], [19, 16], [18, 17], [17, 17], [16, 18], [15, 18],
    [4, 10], [5, 10], [6, 10], [7, 10], [8, 10], [8, 9], [8, 8], [8, 7],
    [20, 10], [19, 9], [18, 10], [17, 9], [16, 10], [16, 11], [16, 12], [16, 13],
    [12, 17], [12, 16], [12, 15], [12, 14], [11, 13], [10, 14], [9, 13], [8, 14],
    [12, 3], [12, 4], [12, 5], [12, 6], [13, 6], [14, 6], [15, 6], [16, 6]
  ]
}
```

Exemple de configuration des murs

Objectifs

Les objectifs sont des zones de scoring : chaque objectif a un id, un nom optionnel et une liste hexes (hexagones qui forment la zone). Ils viennent du scénario (ou de la config du plateau) et sont stockés dans game_state["objectives"]. Ils ne bloquent ni le mouvement ni le tir.

Mouvement et tir

- Les hexagones d'objectif sont des hex normaux : on peut se déplacer dessus, s'y arrêter et tirer depuis ou vers eux. Ils n'apparaissent pas dans wall_hexes et ne sont pas traités comme obstacles dans le BFS ou dans le calcul de ligne de vue.

Contrôle et scoring

- Pour chaque objectif, on regarde quelles unités occupent au moins un hex de sa zone (positions lues dans units_cache).
- Chaque unité a un attribut OC (Objective Control) qui détermine sa contribution au contrôle d'un objectif. On fait la somme des OC de toutes les unités de chaque camp présentes sur les hexes de cet objectif.
- L'objectif est contrôlé par le joueur dont la somme d'OC est strictement supérieure à celle de l'adversaire (en cas d'égalité, pas de contrôle, selon la config tie_behavior).
- En cas de prise de contrôle, tous les hex d'un objectif prennent la couleur du joueur le controlant.
- Le mode de contrôle (ex. sticky : garder le dernier contrôleur tant qu'aucun camp ne mène ; occupy : recalcul à chaque fois) et le tour à partir duquel on compte (ex. start_turn) sont définis dans la config de l'objectif principal (scoring, control).
- Les récompenses d'entraînement et les conditions de victoire peuvent s'appuyer sur le nombre d'objectifs contrôlés (ex. "contrôler plus d'objectifs que l'adversaire").

En résumé : les objectifs sont des zones définies par des hexes dont le contrôle se fait par la comparaison des sommes des OC des unités présentes sur ces hexes pour chaque joueur, sans effet sur le mouvement ni sur la ligne de vue.

```
{
  "objectives_id": "objectives-01",
  "objectives": [
    {"id": 1, "name": "West", "hexes": [[2, 10], [1, 9], [2, 9], [3, 9], [3, 10], [2, 11], [1, 10]]},
    {"id": 2, "name": "North", "hexes": [[12, 1], [12, 2], [11, 1], [11, 0], [12, 0], [13, 0], [13, 1]]},
    {"id": 3, "name": "East", "hexes": [[22, 10], [21, 9], [22, 9], [23, 9], [23, 10], [22, 11], [21, 10]]},
    {"id": 4, "name": "South", "hexes": [[12, 19], [11, 18], [12, 18], [13, 18], [13, 19], [12, 20], [11, 19]]},
    {"id": 5, "name": "Center", "hexes": [[12, 10], [11, 9], [12, 9], [13, 9], [13, 10], [12, 11], [11, 10]]}
  ]
}
```

Exemple de configuration des objectifs

d) Interface de jeu – Unités

1. Icône principale de l'unité

Chaque unité est représentée par une icône (image) centrée sur l'hexagone.

- Source: le champ ICON de l'unité (chemin vers une image au format .webp). Les images sont stockées dans public/icons/.
- Variante joueur 2: pour distinguer les équipes, les unités du joueur 2 utilisent une version à bordure rouge (bleue pour joueur 1) du même fichier : le suffix _red.webp est ajouté au chemin .webp (ex. unit.webp → unit_red.webp).
- Taille: afin d'agrémenter l'immersion, un facteur d'échelle optionnel ICON_SCALE par unité a été implémenté afin que l'icone d'une unité ait une taille correspondant à sa figurine dans le jeu. Les dimensions sont dérivées du rayon de l'hexagone (HEX_RADIUS).

États visuels particuliers de l'icône:

- Unité "fantôme" (replay, déplacement en cours) : icône en semi-transparence avec teinte (couleur de mouvement).
- Unité venant d'être « tuée »: assombrissement (gris, alpha réduit) avant retrait du plateau.
- Cible de tir non valide: en phase de tir, les ennemis qui ne sont pas dans le pool de cibles valides sont affichés en grisé (alpha 0,5, teinte grise).
- Aperçu d'action: en mode aperçu de déplacement, l'icône peut être cliquable pour valider le déplacement ; en aperçu d'attaque, légère transparence.
-

2. Cercle d'éligibilité (activation)

Les unités éligibles à l'activation dans la phase courante sont entourées d'un cercle vert (couleur et épaisseur définies par des variables CSS, ex. --eligible-color, --eligible-outline-width).

En phase combat, les unités ayant chargé ce tour et restant éligibles ont en plus un cercle rouge autour du vert pour les distinguer (priorité de charge).

Ce cercle est purement indicatif ; il ne modifie pas l'icône elle-même.

3. Barre de points de vie (PV)

Au-dessus de l'icône, une barre de PV affiche HP_CUR / HP_MAX (couleur, largeur et hauteur pilotées par des variables CSS). Elle n'est pas dessinée si HP_MAX est absent.

La barre peut s'agrandir et clignoter pour mettre en évidence l'unité ciblée et, en aperçu, refléter les dégâts potentiels (tir ou mêlée).

Multi-cibles: lorsqu'une unité peut tirer sur plusieurs cibles, les barres des cibles valides peuvent participer à un effet de clignotement synchronisé.

4. Logos d'action (phase en cours et replay)

En mode replay (ou lorsque le moteur signale l'unité « en cours d'action »), de petits logos d'action sont affichés en bas à gauche de l'icône (décalage par rapport au centre de l'hexagone). Ils indiquent quelle action est en cours pour cette unité.
Tous ces logos sont des images PNG dans public/icons/Action_Logo/ :

Fichier	Rôle
2 - Movemement.png	**Mouvement**: unité en train de se déplacer (cercle vert).
3 - Shooting.png	**Tir**: unité en train de tirer (carré fond jaune/orange).
3-5 - Advance.png	**Advance**: unité en train d'avancer après tir (carré fond orange).
4 - Charge.png	**Charge**: unité en train de charger (carré fond violet).
5 - Fight.png	**Combat**: unité en train de combattre (carré fond rouge).

Les formes (cercle ou carré arrondi) et les couleurs de fond sont configurées via des variables CSS (ex. --icon-move-bg-color, --icon-shoot-bg-color, etc.).
Ces logos sont affichés uniquement sur l'unité qui effectue l'action (identifiée par movingUnitId, shootingUnitId, chargingUnitId, fightingUnitId, advancingUnitId selon la phase).

5. Indicateur de cible (cible de tir, charge ou combat)

Toute unité qui est la cible d'une action (tir, charge ou combat) affiche un indicateur de cible : un carré arrondi coloré (ex. orange, variable --icon-target-bg-color) avec une icône cible (?) au centre, positionné en bas à gauche de l'icône.
Cas concernés: cible du tireur (shootingTargetId), cible de la charge (chargeTargetId), cible du combat (fightTargetId). Un seul type d'indicateur est utilisé pour tous ces cas (pas d'icône « explosion » distincte dans l'implémentation actuelle).
Taille et style du carré et de l'icône sont pilotés par des variables CSS (--icon-target-size, --icon-target-square-size, bordures, etc.).

6. Icônes d'action en phase de tir (Advance et choix d'arme)

En phase de tir, pour l'unité du joueur courant actuellement en train de tirer (active shooting unit), deux icônes cliquables peuvent apparaître au-dessus de la barre de PV, rendues soit dans le canvas PIXI soit en overlay DOM selon la configuration :

- Advance (3-5 - Advance.png) : proposer l'action « Advance » après le tir (affichée seulement si l'unité peut encore avancer et n'est pas adjacente à un ennemi).
- Choix d'arme (3-1 - Gun_Choice.png) : ouvrir le choix d'arme de tir (affichée si l'unité a au moins une arme utilisable dans ce contexte).

Ces icônes sont positionnées côte à côte (Advance à gauche, choix d'arme à droite) et n'apparaissent que pour l'unité activée au tir et dans le bon mode d'interface (aperçu tir / cible, etc.).

7. Badges de jet (charge et advance)

Pour les jets de dés affichés en replay (ou en direct), des badges sont dessinés à côté de l'unité concernée :

- **Jet de charge:** sur l'unité qui charge (chargingUnitId), un badge affiche le résultat du jet 2D6 (nombre). Couleur du badge : vert (succès) ou rouge (échec), avec bordure et texte lisible.
- **Jet d'advance:** sur l'unité qui avance (advancingUnitId), un badge affiche de la même façon le résultat du jet d'advance.

Ces badges sont des formes (rectangle arrondi) + texte PIXI, positionnés en bas à droite de l'icône, avec un z-index élevé pour rester au-dessus des autres éléments.

8. Compteurs et autres éléments

Compteur d'attaques: en phase combat, les unités en mêlée peuvent afficher le nombre d'attaques restantes (ATTACK_LEFT) dans un petit badge ou texte à côté de l'icône (pour l'unité en cours d'attaque ou les unités éligibles au combat).

Mode debug: optionnellement, un badge d'identifiant (ID d'unité) peut être affiché au centre de l'icône pour identifier plus facilement une unité dans le cadre du débogage.

e) Interface de jeu – le panneau de contrôle

Le panneau de controle regroupe toutes les informations dont peut avoir le joueur pour comprendre tous les aspects de la partie.

La selection du mode de jeu

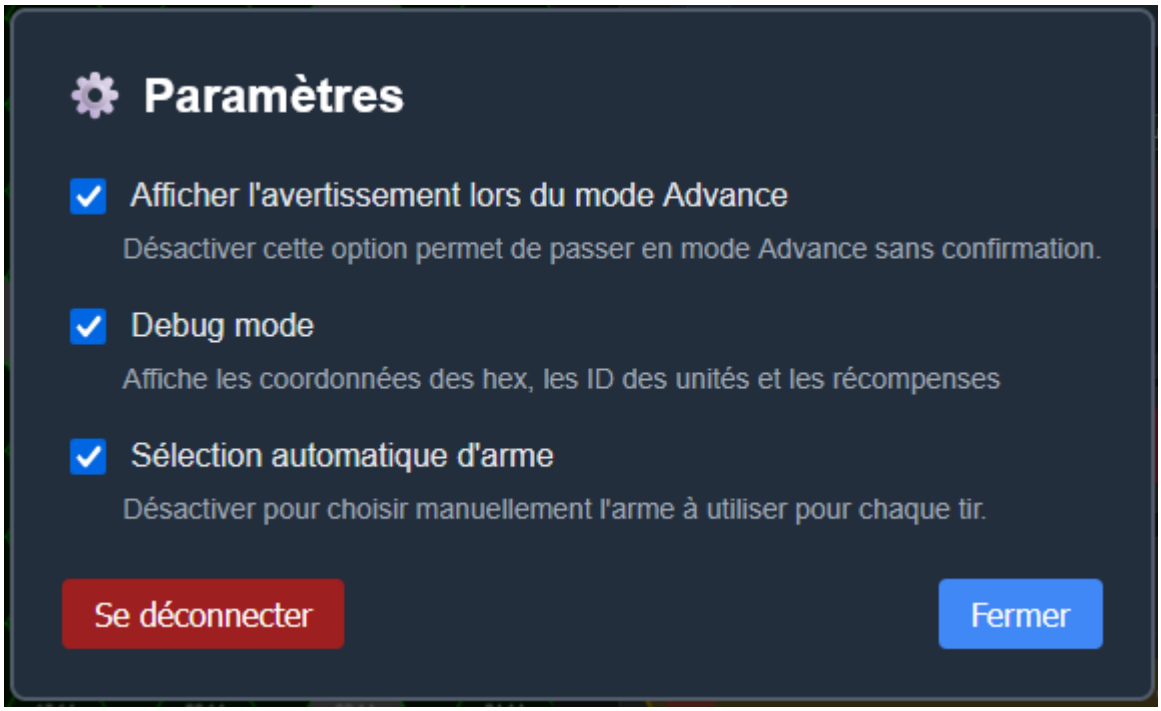
Ces onglets permettent de choisir le mode de jeu.

Les options

Le bouton Option représenté par un rouage fait apparaitre un popup qui propose une selection de paramètres pour le joueur ou l'administrateur.

“Afficher l'avertissement lors du mode Advance” et “Sélection automatique d'arme” sont des options ayant pour but de proposer une granularité de contrôle au joueur.

“Debug mode” est exclusivement réservé au développement. Cette option permet d'afficher les coordonnées des hexagones et les récompenses des actions de l'AI



Turn / Phase tracker

Ce module permet de suivre le déroulé de la partie.
Le jeu dure 5 tours, indiqués à gauche

Chaque tour est divisé en 4 Phases : Mouvement, Tir, Charge et Combat.

Les tours et les phases écoulées sont grisés quand ils ont été effectués pour plus de lisibilité.

Au milieu de ce tracker, nous trouvons unindicateur sur le joueur dont c'est le tour, et l'option pour ce dernier de mettre fin à la phase en cours.

En mode Replay, ces indicateurs (tours, phase, Joueur actif) sont utilisables comme boutons afin de sauter directement vers une action spécifique.

Barre de scoring

La barre de scoring permet de suivre l'évolution du score des joueurs.

Elle affiche le score de chaque joueur et évolue d'un coté ou l'autre selon le ratio de points pour une meilleure visualisation.

Tableau de status des unités

- Bloc par joueur
Chaque joueur a sa propre zone, avec une bande colorée en haut : bleue pour le joueur 1, rouge pour le joueur 2. On y voit un bouton pour replier ou déplier la liste, le libellé du joueur (ex. “Player 1 - Human”, “Player 2 - AI”) et éventuellement les points de victoire.

- Liste des unités
Quand la zone est dépliée, un tableau s’affiche : une ligne par unité vivante. Chaque ligne a un bouton + / − pour afficher ou masquer les armes de l’unité. Ensuite viennent l’identifiant, le nom (avec de petits badges pour les règles spéciales), puis les attributs : Points de vie, Mouvement, Endurance, Sauvegarde, Commandement, Contrôle d’objectif, Valeur. La ligne de l’unité sélectionnée ou sur laquelle on a cliqué est mise en évidence (contour vert). Un clic sur une ligne sélectionne l’unité.

- Détails d’une unité
En dépliant une unité (bouton +), deux blocs apparaissent: un pour les armes à distance, un pour les armes de mêlée. Chaque bloc a un en-tête coloré (bleu pour le tir, rouge pour la mêlée) avec un bouton pour l’ouvrir ou le fermer. En dessous, un tableau liste les armes avec leur nom, leurs règles (en petits badges) et leurs caractéristiques (portée, nombre de tirs, précision, force, pénétration d'armure, dégâts). L’arme actuellement choisie est marquée par un point (bleu ou rose).

- Règles et tooltips
À côté des noms d’unités et d’armes, des pastilles vertes affichent le nom court des règles (ex. « Adrenalised Onslaught »). Au survol d’une pastille, une bulle s’affiche au-dessus avec le texte explicatif de la règle.

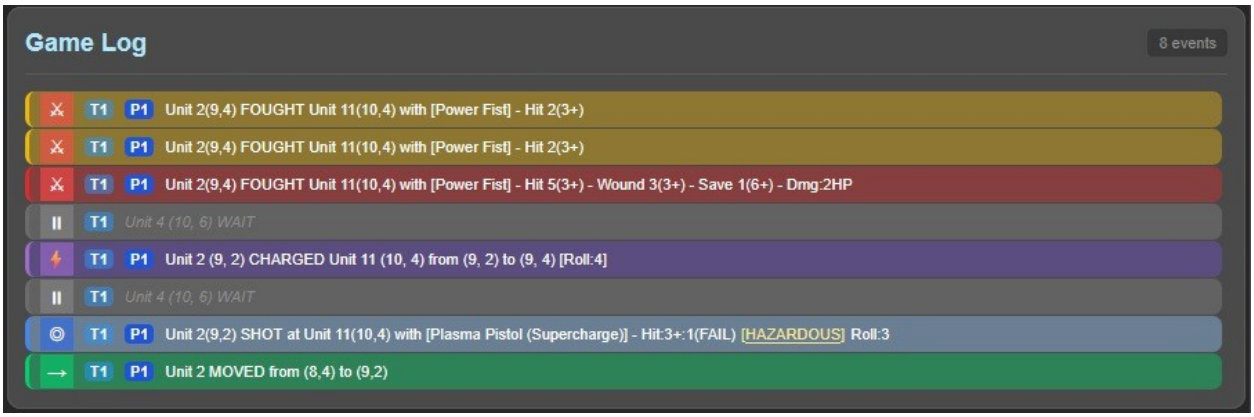
Game log

- Bloc principal
En haut : le titre “Game Log” et à côté le nombre d’événements. En dessous, une liste défilante des derniers événements de la partie, du plus récent en haut.

- Chaque ligne
Une ligne = un événement. On voit de gauche à droite : une petite icône selon le type d’action (déplacement, tir, charge, combat, mort, attente, etc.), éventuellement le numéro de tour ou d’étape, un badge joueur (P1 en bleu, P2 en rouge), puis le texte du message (ex. « P1 Unit 3 moved to (4, 5) »). Les noms de règles dans le message sont en jaune et soulignés ; au survol ou au clic, une bulle apparaît avec la description de la règle (et reste affichée au clic).

- Aspect selon le type
Les lignes ont une bande verticale colorée à gauche et un fond légèrement différent selon le type (attente en gris, contrôle d’objectif teinté bleu/rouge/orange, etc.). Les actions “wait” ou “skip” sont en gris et en italique.

- Vide
S’il n’y a encore aucun événement, un message du type “No events yet. Start playing to see the action log!” s’affiche au centre de la zone.



Réalisation Back-end

Pour permettre au frontend React d'interagir avec le moteur de jeu et les données d'authentification, le backend a été développé en Python, avec le framework Flask et la base de données SQLite.

Python s'est imposé pour plusieurs raisons. D'abord, le moteur de jeu et l'apprentissage par renforcement (Stable-Baselines3, gymnasium) sont en Python. Garder l'API dans le même langage évite des ponts inter-processus et simplifie l'intégration. Ensuite, Python est adapté aux calculs scientifiques et au prototypage rapide, ce qui correspond au cœur métier (simulation tactique, observation, récompenses).

Flask a été choisi pour sa légèreté et son adéquation aux API REST : il permet d'exposer le moteur (démarrage de partie, envoi d'actions, tour IA, reset) et l'authentification sans surcharge.

SQLite a été retenue pour la persistance des comptes utilisateurs et des sessions : elle ne nécessite pas de serveur dédié, convient à un déploiement conteneurisé et à un volume d'utilisateurs limité.

Cette section présente les réalisations backend de l'application. On décrira la structure du code (moteur, handlers de phases, serveur API, base d'authentification) et l'organisation des fichiers. Ensuite, on détaillera les points clés en s'appuyant sur des extraits de code commentés pour illustrer :

- La structure du moteur de jeu (W40KEngine, game_state comme seule source de vérité) et la délégation aux handlers de phases.
- Le point d'entrée step(action) et le déroulement d'une action (décodage, phase courante, mise à jour des caches).
- Les routes jeu (start, action, ai-turn, state, reset) et la synchronisation de l'état renvoyé au frontend (unités, HP depuis units_cache).
- La base SQLite d'authentification (tables utilisateurs, profils, sessions) et le script d'initialisation.
- L'authentification (hachage des mots de passe en PBKDF2-HMAC-SHA256, token bearer de session, vérification du token et du profil sur les routes protégées).
- La validation stricte des données (require_key, require_present) et l'absence de fallback masquant les erreurs.

Une attention particulière sera portée à la sécurité et à la cohérence de l'état de jeu (single source of truth, pas de copie persistante du game_state en dehors du moteur).

1. Structure du code

Le backend de Trazyn's Trials n’est pas organisé en MVC classique : la logique métier de la partie est dans le moteur (engine/), et l’API (services/api_server.py) se contente d’orchestrer le moteur et d’exposer l’état en JSON. La séparation est la suivante :

a) engine/

Le moteur de la simulation. w40k_core.py contient la classe W40KEngine (héritée de gymnasium.Env). Elle gère :

- L'initialisation à partir d’un scénario
- reset()
- step(action)
- L'avancement de phase lorsque les pools d’activation sont vides.
- La délégation de la gestion des phases aux phase handlers.
- Le game_state est un dictionnaire unique ; aucun module ne le duplique ni ne le met en cache de façon persistante (single source of truth).
- Le dossier engine/phase_handlers/ contient un fichier par phase (movement, shooting, charge, fight, deployment, command). Chaque phase construit son pool lors de son initialisation (*_phase_start), traite les actions une par une et met à jour les caches généraux (units_cache pour positions et HP).
- Les modules observation_builder.py, action_decoder.py, reward_calculator.py et game_state.py fournissent l’observation, le décodage d’actions, les récompenses et la gestion des unités et du déploiement.

b) shared/

Utilitaires partagés, notamment data_validation.py (require_key, require_present) utilisés dans le moteur et l’API pour toute donnée requise ; en cas de clé ou valeur manquante, une erreur explicite est levée (pas de valeur par défaut pour éviter une erreur, coding_practices.mdc).

c) config/

Fichiers de configuration (scénarios, règles, agents). La base SQLite des utilisateurs est dans config/users.db ; le script de création des tables est exécuté au démarrage de l'API (initialize_auth_db).

d) services/

services/api_server.py est le fichier principal de l'application backend : il crée l'application Flask, configure CORS, enregistre les blueprints ou routes (jeu, auth, replay, health), se connecte à SQLite, initialise ou réutilise une instance de W40KEngine, et lance le serveur.

Avant d'envoyer l'état au frontend, les HP des unités sont synchronisés depuis units_cache (_sync_units_hp_from_cache), afin que le client reçoive toujours la vérité serveur.

e) Préparation des règles et séquençage

Avant le développement des handlers de phase, les règles métier (ordre des phases, éligibilité des unités, restrictions de mouvement et de tir, etc.) ont été formalisées dans un document de spécification (*Documentation/AI_TURN.md)*. Ce document contient un arbre de décision pour chaque phase : conditions d'éligibilité, actions autorisées, mise à jour des pools et des caches, et fin de phase. Les handlers du moteur (*engine/phase_handlers/*) ont été conçus pour suivre strictement ce séquençage, afin de garantir la conformité du comportement (parties humaines, replay et entraînement IA) et de permettre la vérification automatique via *scripts/check_ai_rules.py* et *ai/analyzer.py.*

Le décision tree ayant servi à la modélisation de la phase de mouvement est en **Annexe 4**.

2. Base de données

L’application utilise SQLite pour le stockage des données d’authentification et des droits.

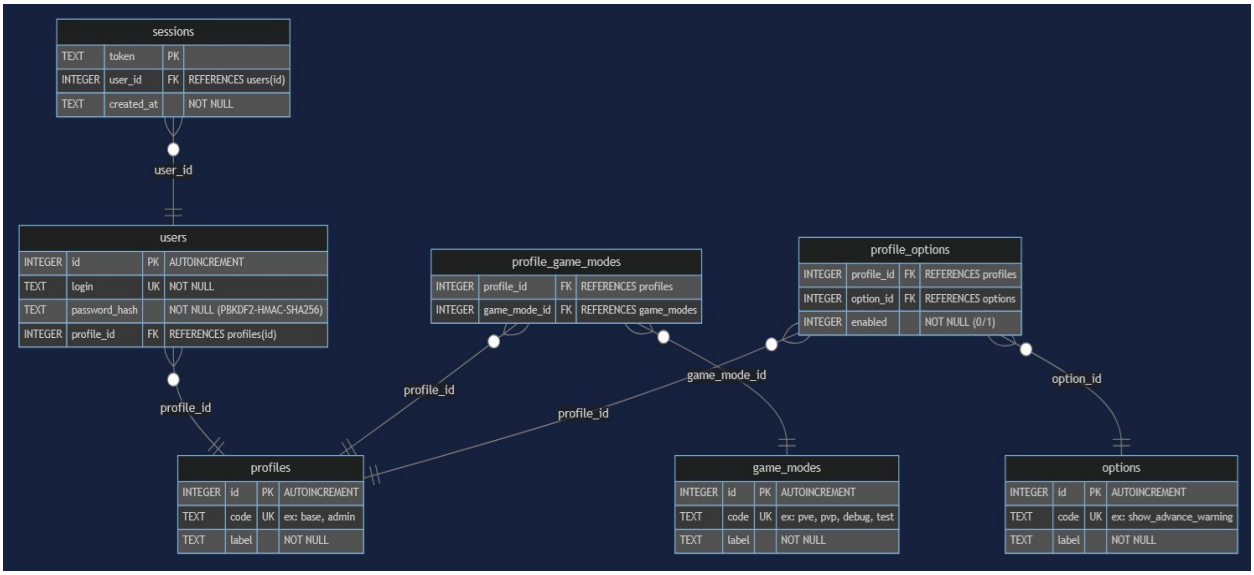
Ce choix est motivé par la nature du projet (démo, formation, déploiement maîtrisé) : pas de serveur de base dédié, un seul fichier users.db, simplicité des sauvegardes et déploiement en conteneur. Les données à persister sont structurées (utilisateurs, profils, modes de jeu, options, sessions) ; un modèle relationnel convient. SQLite est supporté nativement par Python et ne nécessite pas de service externe.

Les données sont organisées en tables principales :

- users (identifiant, mot de passe haché, lien vers profil), profiles (rôles, ex. base / admin)
- game_modes et options (modes de jeu et options accessibles).
- profile_game_modes et profile_options (association profils–modes et profils–options).
- sessions (tokens de session pour l’authentification des requêtes).

Les mots de passe sont hachés en PBKDF2-HMAC-SHA256 (200 000 itérations) :ils ne sont jamais stockés en clair.

Le script complet de création et d’amorçage des tables est donné en **Annexe 5.**



Modèle BDD – Base d'authentification (config/users.db)

3. API RESTful

L'API expose des routes REST pour que le frontend React puisse démarrer une partie, envoyer des actions, déclencher le tour IA, récupérer l'état, se réinitialiser, s'authentifier et consulter les replays.

Jeu

- POST /api/game/start (démarrage avec scénario/mode)
- POST /api/game/action (action joueur)
- POST /api/game/ai-turn (tour IA)
- GET /api/game/state (état courant)
- POST /api/game/reset (reset de la partie).

Les réponses contiennent le game_state sérialisé en JSON (unités, phase, tour, plateau, objectifs, etc.), avec HP et positions alignés sur units_cache.

Authentification

- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/me.

La connexion repose sur un token bearer de session ; les routes protégées (jeu, replay, debug) vérifient ce token et le profil ; une action non autorisée renvoie 403.

Replay et santé

Routes pour lister et récupérer les fichiers de replay (avec protection path traversal) et GET /api/health pour le healthcheck (ex. Docker).

Un tableau récapitulatif (méthode, URL, paramètres, codes de retour) figure en **Annexe 6**.

4. Composants métier

Le cœur métier du jeu est dans le moteur ; l'API délègue à celui-ci et formate les réponses.

a) Point d'entrée step(action) et délégation

La méthode step(action) de W40KEngine (dans w40k_core.py) :

- Décode l'action via ActionDecoder (index discret → type d'action, unité, cible, destination, etc.) en s'appuyant sur le masque d'actions valides (MaskablePPO).
- Détermine la phase courante (move, shoot, charge, fight, etc.).
- Appelle le handler de phase approprié (ex. movement_handlers.handle_movement_phase_step).

.

Le handler :

- Consomme une activation (une unité effectue une action)
- Met à jour le game_state et les caches (update_units_cache_position, update_units_cache_hp ; en cas de mort, retrait du cache).

La fin de phase est décidée par éligibilité : lorsque les pools d'activation sont vides, on passe à la phase suivante.

Aucune mise à jour d'état n'est faite en dehors de ces handlers ; la cohérence est garantie par les règles documentées dans AI_TURN.md et vérifiées par scripts/check_ai_rules.py.

b) Routes jeu et synchronisation avec le frontend

Les routes jeu instancient ou réutilisent un W40KEngine, appellent reset() ou step(action) avec les paramètres reçus (body JSON), puis renvoient l'état. Avant envoi, les unités sont synchronisées avec units_cache pour les HP et positions (_sync_units_hp_from_cache) afin que le client reçoive une vue cohérente avec la seule source de vérité côté serveur.

c) Authentification et sécurité

L’inscription et la connexion sont gérées dans api_server.py.

À l’inscription, le mot de passe est haché (PBKDF2-HMAC-SHA256, 200 000 itérations) avant stockage.

À la connexion, le mot de passe fourni est comparé au hash. En cas de succès, un token de session est généré et renvoyé (et éventuellement stocké en base pour vérification).

Les routes protégées utilisent un middleware (ou une fonction) qui lit le token dans l’en-tête Authorization, vérifie sa validité et le profil, et renvoie 401 ou 403 en cas d’échec.

Les secrets (clé de session, chemins sensibles) ne sont pas codés en dur. En production (ex. déploiement Synology), des variables d’environnement sont utilisées.

d) Validation des données et gestion des erreurs

Toute donnée requise (paramètres de requête, body, champs du game_state ou des configs) est accédée via require_key ou require_present (shared/data_validation.py).

En cas d’absence, une exception est levée et transformée en réponse d’erreur explicite (4xx). Aucun fallback silencieux n’est utilisé pour masquer une donnée manquante (AI_TURN.md, coding_practices.mdc).

Les erreurs serveur sont traitées de façon à ne pas exposer d’informations sensibles tout en renvoyant un code et un message adaptés.

Des extraits de code significatifs (point d’entrée step, handler de phase, appel API) sont regroupés en **Annexe 7.**

Eléments de sécurité

La sécurité de l'application Trazyn's Trials a été prise en compte à la fois côté API (Flask, moteur Python) et côté frontend (React), en cohérence avec les règles du projet (pas de fallback, validation explicite).

1. Authentification et autorisation

- **Token de session** : L'authentification repose sur un token porteur (bearer) stocké côté serveur dans la table sessions.
    - À la connexion, un token est généré (par ex. secrets.token_urlsafe(48)) et enregistré en base avec l'identifiant utilisateur. Il est renvoyé au client dans la réponse de login.
    - Le frontend l'envoie dans l'en-tête Authorization: Bearer <token> pour les requêtes protégées.
    - Côté API, il n'y a pas de middleware Flask global : chaque route protégée appelle en début de handler la fonction utilitaire _get_authenticated_user_or_response(), qui utilise _extract_bearer_token() pour lire et valider le format de l'en-tête, puis interroge la table sessions (avec jointure sur users et profiles).
    - Si le token est absent, invalide ou inconnu, la fonction renvoie une réponse 401 ; sinon elle fournit l'utilisateur authentifié (id, login, profile_id, profile_code) au handler.
    - Les routes jeu, replay et modes Debug/Test suivent ce schéma.

- **Hachage des mots de passe** : Les mots de passe sont hachés avec PBKDF2-HMAC-SHA256 (200 000 itérations) avant stockage dans users.password_hash.
    - Ils ne sont jamais conservés en clair
    - En cas de compromission de la base, les mots de passe restent protégés.

- **Protection des routes** : Les routes sensibles (démarrage de partie, envoi d'action, tour IA, replay, modes Test/Debug) appellent _get_authenticated_user_or_response() puis, pour les routes jeu, vérifient que le mode demandé est autorisé pour le profil via _is_mode_allowed() (droits issus de profile_game_modes). Si le mode n'est pas autorisé, la route renvoie 403.
    - Côté frontend, le routage React et la résolution des permissions (modes et options selon le profil) limitent l'affichage des écrans et des actions aux seuls utilisateurs autorisés.
    - La protection effective reste côté serveur.

2. Protection contre les failles XSS

- **Côté API** :
  - les réponses sont en JSON.
  - Il n'y a pas de rendu HTML côté serveur, ce qui limite la surface d'attaque XSS.

- **Côté frontend** :
  - Le contenu affiché provient de l'état de partie et des réponses API.
  - React échappe par défaut le contenu rendu, ce qui réduit les risques XSS.
  - Toute utilisation de *dangerouslySetInnerHTML* ou de rendu de contenu utilisateur non contrôlé (par ex. messages de log ou champs libres) augmenterait le risque et doit être évitée ou strictement sanitée.

3. Validation des données et injection

- **Backend / moteur**:
  - Toute donnée requise est accédée via *require_key* et *require_present* (*shared/data_validation.py*).
  - En cas de clé ou valeur manquante, une erreur explicite est levée.
  - Aucun fallback ne masque une donnée manquante (*AI_TURN.md*, *coding_practices.mdc*).
  - Les configurations (scénarios, training, rewards) sont chargées avec vérification des clés attendues ; les erreurs de config ne sont pas masquées.

- **Injection SQL:**
  - Le moteur de jeu ne dialogue pas directement avec la base.
  - Les requêtes SQL (auth) sont paramétrées (placeholders).
  - Aucune concaténation de chaînes utilisateur dans des requêtes SQL.

- **Injection dans le moteur:**
  - Les paramètres d'action (body JSON) sont interprétés par le moteur après décodage ; la validation stricte et l'absence de valeurs par défaut anti-erreur limitent les comportements indéterminés.

4. Stockage sécurisé des informations sensibles

- Les mots de passe ne sont jamais stockés en clair. Seuls les hash sont enregistrés dans *config/users.db*.
- Les secrets (clé de session, chemins sensibles) ne sont pas codés en dur . Le déploiement Synology utilise des variables d'environnement (*Deployment_Synology.md*).
- Protection path traversal: Les endpoints replay qui acceptent un chemin ou un nom de fichier valident que la cible reste dans le répertoire autorisé, afin d'éviter l'accès à des fichiers hors périmètre.

Jeux d'essai

Afin de garantir le bon fonctionnement de l'application Trazyn's Trials et la conformité du comportement aux spécifications (règles de jeu, moteur, API, frontend, pipeline IA), une série de tests a été mise en place. Les vérifications reposent sur :

- Des tests manuels (démarrage de partie, envoi d'actions, tour IA, accès aux modes selon le profil)
- Des outils automatisés :
  - scripts/check_ai_rules.py (conformité du code aux règles AI_TURN.md et coding_practices.mdc).
  - ai/analyzer.py (analyse du fichier step.log produit en entraînement avec --step, détection des violations par phase et métriques d'usage des règles).
  - ai/hidden_action_finder.py (actions effectuées mais non loguées).
  - ai/bot_evaluation.py (évaluation contre RandomBot, GreedyBot, DefensiveBot, suivi des win rates et métrique bot_eval_combined).

Exemple : Analyzer.py + vérification visuelle

Processus de validation de la conformité du moteur aux règles métier via l'analyse des logs d'entraînement (ai/analyzer.py) et une vérification visuelle (partie ou replay).

a) Données en entrée

- **Commande de génération du log**
  - python ai/train.py --agent Infantry_Troop_RangedTroop --training-config default --rewards-config Infantry_Troop_RangedTroop --scenario default --test-only --macro-eval-mode micro --test-episodes 100 --step
- **Fichier produit**
  - step.log (traces détaillées de chaque step : phase, action, unité, cible, etc.)
- **Commande** d'analyse
  - python ai/analyzer.py step.log

- **Résultats attendus**
  - Analyzer : 0 violations pour les phases mouvement, tir, charge, combat (comportement conforme à AI_TURN.md).
  - Vérification visuelle : lors d'une partie PvE ou d'un replay, les phases s'enchaînent correctement, les unités ne font pas d'actions invalides (ex. pas de tir sans ligne de vue, pas de mouvement vers hex adjacent ennemi).

b) Résultats obtenus

- **Step.log**
  - Voir **Annexe 8**.

- **analyzer.log**
  - Voir **Annexe 9**.

c) Analyse

- **Step.log**
  - Le fichier est présent
  - La partie va jusqu'à son terme.
  - Les deux joueurs agissent.
  - Toutes les phases sont exécutées.

- **analyzer.log**
  - Aucune erreur signalée dans le Summary
  - Vérification manuelle de metrics :
    - Nombre d'épisodes joués selon argument (100) avec durée et nombre d'actions.
    - Vérification de la crédibilité du Ratio et des types de victoires.
    - Présence de métrics pour des règles spéciales conformes et attendues.

- **Vérification visuelle**
  - Si une erreur apparait :
    - Rejouer le scenario concerné dans le mode Replay et reproduire l'action visée afin de confirmer le problème metier ou vérifier si l'analyzer n'a pas généré une fausse erreur.

Respect du RGPD

Le Règlement général sur la protection des données (RGPD) encadre le traitement des données personnelles. L'application Trazyn's Trials, dans sa version actuelle, prend en compte les principes suivants :

Minimisation des données :

- L'application ne collecte que les données nécessaires à l'authentification et au contrôle d'accès.
- Lors de l'inscription, les informations demandées sont limitées à un identifiant de connexion (login) et un mot de passe (haché avant stockage, jamais conservé en clair).
- Le profil (base ou admin) et les droits associés (modes de jeu, options) sont gérés en base (tables users, profiles, profile_game_modes, profile_options).
- Aucune donnée de paiement, aucun numéro de téléphone ni adresse e-mail n'est collecté dans la version actuelle.

Information des utilisateurs :
Les utilisateurs peuvent être informés de l'utilisation de leurs données via une page ou une politique de confidentialité (à finaliser).

Droits des utilisateurs :
Droit d'accès et de rectification via la gestion du compte (consultation et mise à jour des informations d'authentification). Le droit à l'effacement (suppression de compte) peut être prévu. Les droits à l'opposition, à la limitation et à la portabilité sont à traiter selon les évolutions fonctionnelles. Aucune donnée n'est utilisée à des fins de profilage ou de marketing.

Sécurité des données :
Voir la section « Éléments de sécurité » (hachage PBKDF2-HMAC-SHA256, token de session, validation stricte, protection des routes, pas de fallback masquant les erreurs).

Cookies :
L'application n'utilise que les cookies strictement nécessaires au fonctionnement (session, authentification). Aucun cookie de suivi ou publicitaire n'est utilisé par défaut.

Déploiement

L’application Trazyn's Trials est déployée sur un NAS Synology (DSM) via Docker Compose, ce qui permet un hébergement maîtrisé et une démonstration en conditions réelles. Le déploiement est décrit en détail dans Documentation/Deployment_Synology.md.

La configuration retenue comporte deux conteneurs :

- Un service pour le backend Flask (API REST, moteur de jeu, authentification), exposé en interne sur le port 5001
- Un service pour le frontend (fichiers statiques de l’application React/TypeScript/Vite servis par Nginx), exposé en interne sur le port 80 et mappé sur le port 8081 de l’hôte.

Le reverse proxy du NAS (HTTPS, par ex. via DDNS) reçoit les requêtes des utilisateurs et les achemine vers le frontend.

Les appels vers l’API sont proxifiés (par ex. /api/* vers le backend).

Les variables d’environnement obligatoires pour les volumes (SYNO_CONFIG_PATH, SYNO_MODELS_PATH, SYNO_RUNTIME_PATH) sont documentées

Aucun fallback n’est utilisé pour ces chemins.

Les volumes persistants concernent la base d’authentification (users.db), les modèles IA (ai/models/) et le répertoire runtime (logs).

Le répertoire config/ n’est pas entièrement monté afin de ne pas écraser les configurations du dépôt.

Les points de vigilance (compatibilité des dépendances Python, appels API en URL relative pour éviter les problèmes CORS, healthcheck /api/health utilisé dans le Compose) sont décrits dans la documentation de déploiement.

L’URL d’accès dépend de la configuration du reverse proxy et du DDNS (ex. *https://game.40k-greg.synology.me*).

Veille durant le projet

Une veille technologique a été menée sur les thèmes suivants, en lien avec les choix d'implémentation du projet :

- **Apprentissage par renforcement et action masking:** Utilisation de MaskablePPO (sb3-contrib) pour restreindre les actions aux seules actions valides (masque binaire), en cohérence avec les règles métier du jeu : réduction des actions invalides et accélération de l'apprentissage. Sources : documentation Stable-Baselines3 et articles sur l'action masking en RL.
- **Évaluation robuste des agents**: Mise en place de bots de référence (Random, Greedy, Defensive) et d'une métrique composite (*bot_eval_combined*) pour mesurer la performance réelle et limiter le surajustement à un seul type d'adversaire. Référence : *AI_TRAINING.md*, *AI_METRICS.md*.
- **Métriques et tuning PPO**: Suivi des métriques d'entraînement (loss, explained_variance, clip_fraction, approx_kl, entropy, gradient_norm, etc.) et guide de tuning pour identifier et traiter plateaux, effondrements et instabilités. Référence : *AI_METRICS.md*.
- **Sécurisation backend**: Bonnes pratiques (hachage fort, token de session, validation stricte, pas de fallback silencieux), recommandations OWASP et documentation Flask.
- **Documentation et conformité du code**: Règles de tour (*AI_TURN.md*), architecture (*AI_IMPLEMENTATION.md*), scripts de vérification automatique (*check_ai_rules*, *analyzer*) pour maintenir la cohérence entre code et règles métier.

Déploiement Docker sur NAS Synology

L'application Trazyn's Trials est déployée sur un NAS Synology (DSM) via Docker Compose, ce qui permet un hébergement maîtrisé et une démonstration en conditions réelles. Le déploiement est décrit en détail dans *Documentation/Deployment_Synology.md*.

Configuration retenue

- **Backend**: service Flask (API REST, moteur de jeu, authentification), exposé sur le port 5001 dans le conteneur et mappé sur le port 5001 de l'hôte.
- **Frontend**: application React/TypeScript/Vite buildée puis servie en statique par Nginx, port 80 dans le conteneur, mappé sur le port 8081 de l'hôte.
- **Reverse proxy** : le reverse proxy DSM reçoit les requêtes des utilisateurs et les achemine vers le frontend ; les requêtes vers /api/* sont proxyfiées par Nginx vers le backend. Les ports 8081 et 5001 ne sont pas exposés directement sur Internet.
- **Variables d'environnement**: les chemins des volumes sont obligatoires (*SYNO_CONFIG_PATH*, *SYNO_MODELS_PATH*, *SYNO_RUNTIME_PATH*) ; aucune valeur par défaut n'est utilisée (le Compose lève une erreur si une variable est absente).

Volumes persistants

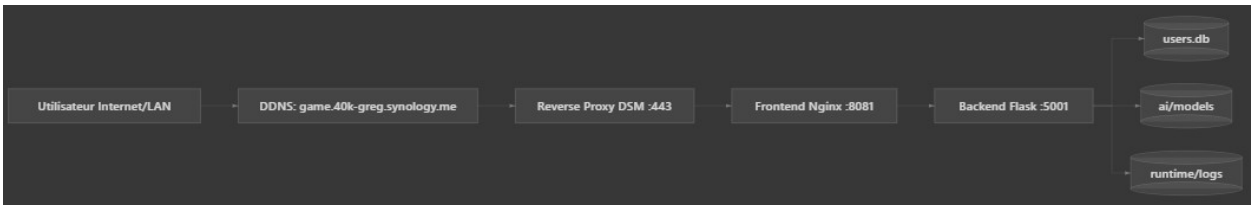
- **Base d'authentification**: *users.db* (montée depuis *SYNO_CONFIG_PATH*).
- **Modèles IA**: *ai/models/* (monté depuis *SYNO_MODELS_PATH*).
- **Runtime** : logs et artefacts (monté depuis *SYNO_RUNTIME_PATH*).

Le répertoire *config/* n'est pas entièrement monté afin de ne pas écraser les fichiers de configuration du dépôt (scénarios, règles, etc.).

Points de vigilance

- Compatibilité des dépendances Python (*requirements.runtime.txt*, ex. numpy pour le chargement des modèles PPO).
- Appels API en URL relative (*/api*) côté frontend pour éviter les problèmes CORS avec le proxy.
- Healthcheck backend : le Compose utilise *GET /api/health* sur le backend pour vérifier que le service est prêt avant de considérer le frontend comme démarré.

L'URL d'accès dépend du reverse proxy et du DDNS : *https://game.40k-greg.synology.me*



Conclusion

Ce projet m'a permis de realiser une application complète combinant :

- Un moteur (phases mouvement, tir, charge, combat, deploiement) fidèle au métier.
- Un backend structuré et source de vérité.
- Une API.
- Un frontend réactif et piloté.
- Un entrainement IA en cours de perfectionnement.
- Une architecture modulaire.
- Une validation des donnees stricte.
- Une documentation structurée.

Perspectives:

- Finaliser le macro agent.
- Optimiser les agents.
- Ajouter un tutoriel.
- Ajouter la gestion des escouades.
- Enrichir les factions et unités disponibles.

Annexes

Annexe 1 : Liste des librairies

Backend		
Librairie	**Version**	**Rôle / description**
Flask	03/01/02	Framework web : API REST (routes jeu, auth, replay, health).
Flask-CORS	6.0.1	Gestion des en-têtes CORS pour les appels depuis le frontend.
numpy	2.4.2 / 2.2.3*	Calculs numériques (observations, récompenses, masques d'actions).
torch	02/05/01	Inférence des modèles IA (PvEController), entraînement (SB3 sous le capot).
torchaudio	02/05/01	Dépendance de l'écosystème PyTorch (pas d'usage direct dans le code).
torchvision	0.20.1	Idem, dépendance PyTorch.
gymnasium	01/01/01	Environnement RL : moteur (W40KEngine) hérite de gym.Env.
stable-baselines3	02/06/00	Algorithmes RL (PPO, DQN, callbacks, VecEnv, Monitor).
sb3-contrib	02/06/00	MaskablePPO et ActionMasker pour les actions contraintes.

* requirements.runtime.txt indique pandas 2.2.3 ; la version numpy peut varier selon le fichier.

Utilisées en plus dans le code mais pas dans requirements.runtime.txt :

tensorboard : utilisée via torch.utils.tensorboard.SummaryWriter dans ai/metrics_tracker.py (donc fournie par torch).

tqdm : barres de progression dans ai/train.py et ai/multi_agent_trainer.py (à ajouter en dépendance si tu veux une liste complète).

Frontend		
Librairie	**Version**	**Rôle / description**
react	^19.1.1	Bibliothèque UI : composants, hooks, état.
react-dom	^19.1.1	Rendu React dans le DOM (y compris createPortal dans GameLog).
react-router-dom	^7.8.2	Routage (Routes.tsx) : /auth, /game, /replay, redirections, guards.
pixi.js-legacy	^7.4.3	Rendu 2D du plateau hex, unités, objectifs (BoardDisplay, UnitRenderer).

DevDependencies (build / outil, pas dans le bundle final exposé) :

vite (^7.1.2) : build et dev server.

typescript (~5.8.3) : typage.

@vitejs/plugin-react (^5.0.0) : support React pour Vite.

tailwindcss (^4.1.13), postcss, autoprefixer : styles.

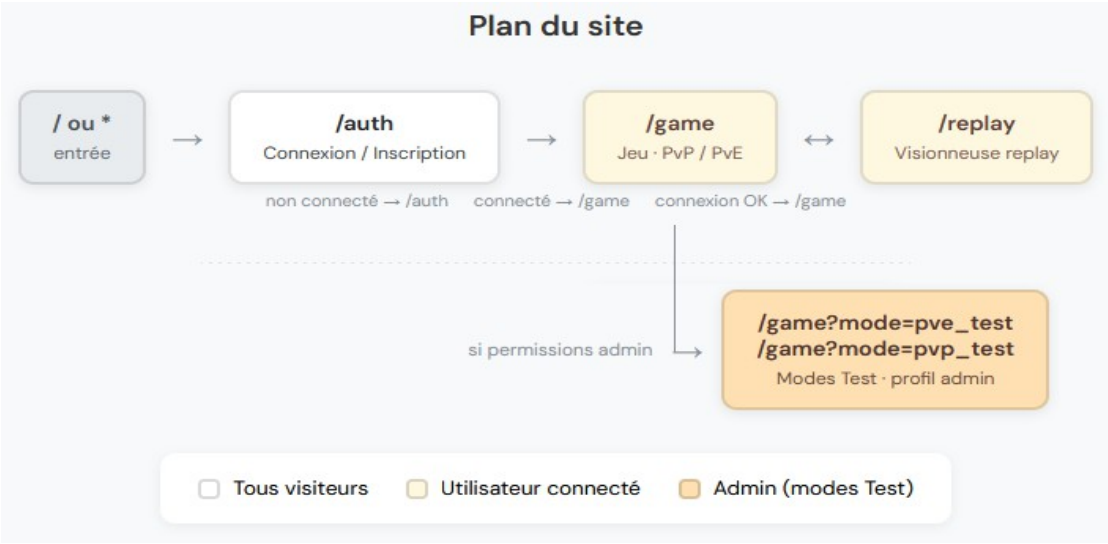
eslint, typescript-eslint, @biomejs/biome : lint / format.

@types/react, @types/react-dom : types pour React.

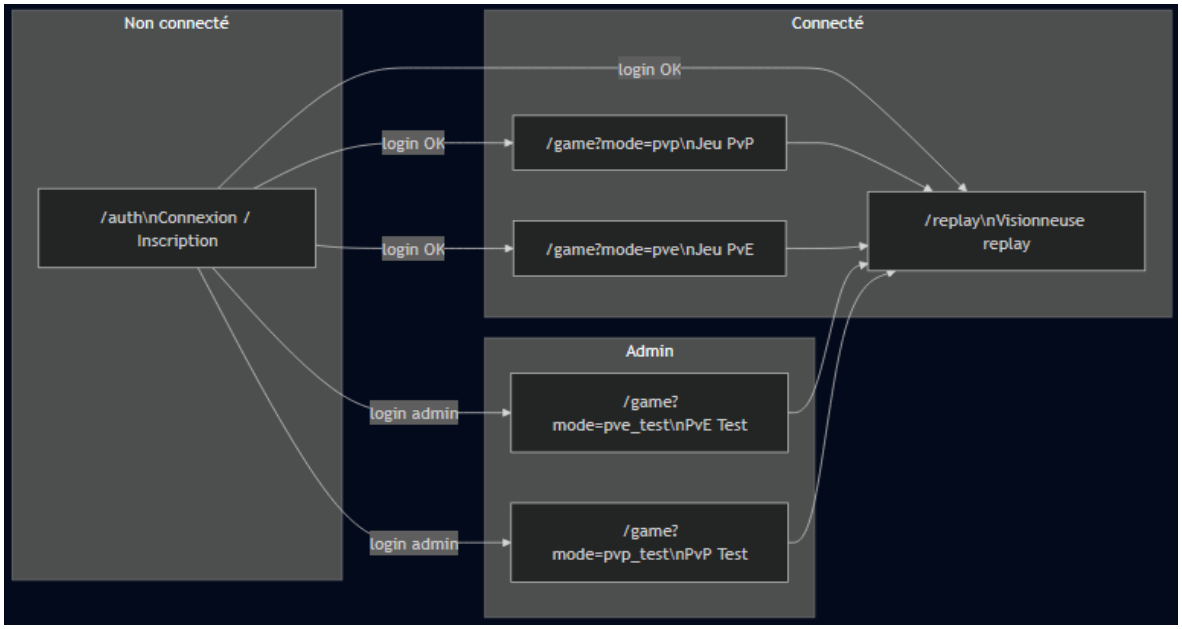
Annexe 2 : Composants

Composant	Fichier	Description / fonctionnalités
App (Routes)	src/Routes.tsx	Point d'entrée du routeur : routes /auth, /game, /replay, /. Guards (ProtectedGameRoute, ProtectedReplayRoute, RootRedirect) : vérification de session et permissions (modes de jeu), redirection vers /auth si non connecté.
AuthPage	pages/AuthPage.tsx	Page de connexion et d'inscription. Formulaire login/register, appel POST /api/auth/login, sauvegarde de la session (token, user, permissions), redirection après succès.
BoardWithAPI	components/BoardWithAPI.tsx	Écran de jeu principal (PvP / PvE). Utilise useEngineAPI (start, action, ai-turn, state), gère le state de partie, les choix de règles (deploy, etc.), orchestre plateau, log, barre de phase et menu. Affiche BoardPvp + SharedLayout (GameLog, UnitStatusTable, TurnPhaseTracker, SettingsMenu).
BoardReplay	components/BoardReplay.tsx	Visionneuse de replay. Charge et parse un fichier replay (step.log ou équivalent), rejoue les actions pas à pas, réutilise SharedLayout + BoardPvp en mode lecture seule avec contrôles play/pause/step.
BoardPvp	components/BoardPvp.tsx	Plateau de jeu partagé (jeu et replay). Gère clics (sélection d'unité, mouvement, tir, charge, combat), prévisualisations (déplacement, tir, charge), appelle drawBoard (BoardDisplay) et renderUnit (UnitRenderer), affiche WeaponDropdown. Utilisé par BoardWithAPI et BoardReplay.
BoardDisplay	components/BoardDisplay.tsx	Rendu PIXI du plateau hex : grille, couleurs des cellules, objectifs, murs, surbrillance (déplacement, tir, charge). Exporte drawBoard(config, …) utilisé par BoardPvp.
UnitRenderer	components/UnitRenderer.tsx	Rendu PIXI d'une unité : sprite, barre de vie, icônes (armes, avance, etc.), états (sélection, preview, cible). Exporte renderUnit(unit, centerX, centerY, app, …) utilisé par BoardPvp. Gère aussi les indicateurs en mode replay (explosion, charge, combat).
WeaponDropdown	components/WeaponDropdown.tsx	Menu contextuel de sélection d'arme (tir ou corps-à-corps). Liste des armes disponibles avec infos (règles, tooltips), clic pour choisir, fermeture au clic extérieur.
GameLog	components/GameLog.tsx	Affichage du log d'actions/événements. Reçoit une liste d'événements, affiche messages, timestamps, types (mouvement, tir, charge, combat, etc.), tooltips sur les règles [rule_id]. Utilisé par BoardWithAPI, BoardReplay et GameController (ce dernier n'étant pas dans le flux actif).
SettingsMenu	components/SettingsMenu.tsx	Menu / modal des réglages : déconnexion, options (avertissement advance, mode debug, sélection auto d'arme) selon les permissions. Ouvert depuis SharedLayout (icône paramètres).
SharedLayout	components/SharedLayout.tsx	Layout commun jeu / replay : colonne gauche (plateau ou viewer), colonne droite (log, tableaux d'unités, phase). Barre de navigation (PvP, PvE, PvE Test, PvP Test, Replay) avec état actif selon l'URL, bouton paramètres. Enveloppe les enfants dans ErrorBoundary.
TurnPhaseTracker	components/TurnPhaseTracker.tsx	Affichage et contrôle du tour / phase : tour courant, phase courante, liste des phases, joueur actif (P1/P2). En mode replay : boutons pour avancer tour/phase/joueur. En jeu : bouton "Fin de phase" si autorisé.
UnitStatusTable	components/UnitStatusTable.tsx	Tableau des unités par joueur (P1 / P2) : nom, PV, caractéristiques, règles d'unité et d'arme (tooltips). Sélection d'unité, mise en évidence de l'unité guidée / cliquée. Peut afficher les points de victoire. Utilisé dans BoardWithAPI, BoardReplay et GameController.
ErrorBoundary	components/ErrorBoundary.tsx	Composant de capture d'erreurs React : affiche un fallback (ou un message par défaut) en cas d'erreur dans l'arbre des enfants, optionnellement appelle onError. Utilisé par SharedLayout, BoardWithAPI, BoardReplay, PlayerVsAIPage (cette page n'étant pas dans le flux actif).

Annexe 3 : Plan du site et Roadmap



/ et routes inconnues *: redirection vers /auth si non connecté, vers /game?mode=pve (ou mode autorisé) si connecté.Plan du site – Accès : tous (/auth), connecté (/game, /replay), admin (modes Test).*



Enchaînement des écrans – Parcours utilisateur (auth → jeu → replay)

Annexe 4 : Decision Tree

```
START OF THE PHASE
For each unit
├── ❌ Remove Mark units_moved (done in command_phase_start)
├── ❌ Remove Mark units_fled (done in command_phase_start)
├── ❌ Remove Mark units_shot (done in command_phase_start)
├── ❌ Remove Mark units_charged (done in command_phase_start)
├── ❌ Remove Mark units_fought (done in command_phase_start)
│
├── ELIGIBILITY CHECK (move_activation_pool Building Phase)
│   ├── unit.HP_CUR > 0?
│   │   └── NO → ❌ Dead unit (Skip, no log)
│   ├── unit.player === current_player?
│   │   └── NO → ❌ Wrong player (Skip, no log)
│   ├── Has at least one valid adjacent hex (not occupied, not adjacent to enemy, not a wall)?
│   │   └── NO → ❌ Unit cannot move (Skip, no log)
│   └── ALL conditions met → ✅ Add to move_activation_pool
│
├── STEP : UNIT_ACTIVABLE_CHECK → is move_activation_pool NOT empty ?
│   ├── YES → Current player is an AI player ?
│   │   ├── YES → pick one unit in move_activation_pool
│   │   │   └── Valid destination exists (BFS pathfinding OK, within MOVE, NOT through/into wall hexes, NOT through/into adjacent to enemy hexes) ?
│   │   │       ├── YES → MOVEMENT PHASE ACTIONS AVAILABLE
│   │   │       │   ├── 🎯 VALID ACTIONS: [move, wait]
│   │   │       │   ├── ❌ INVALID ACTIONS: [shoot, charge, attack] → end_activation (ERROR, 0, PASS, MOVE, 1, 1)
│   │   │       │   └── AGENT ACTION SELECTION → Choose move ?
│   │   │       │       ├── YES → ✅ VALID → Execute move action
│   │   │       │       │   ├── The active_unit was adjacent to an enemy unit at the start of its move action ?
│   │   │       │       │   │   ├── YES → end_activation (ACTION, 1, FLED, MOVE, 1, 1)
│   │   │       │       │   │   └── NO → end_activation (ACTION, 1, MOVE, MOVE, 1, 1)
│   │   │       │       └── NO → Agent chooses: wait?
│   │   │       │           ├── YES → ✅ VALID → Execute wait action
│   │   │       │           │   └── end_activation (WAIT, 1, PASS, MOVE, 1, 1)
│   │   │       │           └── NO → Agent chooses invalid action (shoot/charge/attack)?
│   │   │       │               └── ❌ INVALID ACTION ERROR → end_activation (ERROR, 0, PASS, MOVE, 1, 1)
│   │   │       └── NO → end_activation (NO, 0, PASS, MOVE, 1, 1)
│   │   │
│   │   └── NO → Human player → STEP : UNIT_ACTIVATION
│   │       ├── If any, cancel the Highlight of the hexes in valid_move_destinations_pool
│   │       ├── Player activate one unit by left clicking on it
│   │       └── Build valid_move_destinations_pool (NOT wall hexes, NOT adjacent to enemy hexes, reacheable using BFS pathfinding within MOVE attribute distance)
│   │           └── valid_move_destinations_pool not empty ?
│   │               ├── YES → STEP : PLAYER_ACTION_SELECTION
│   │               │   ├── Highlight the valid_move_destinations_pool hexes by making them green
│   │               │   └── Player select the action to execute
│   │               │       ├── Left click on a hex in valid_move_destinations_pool → Move the unit's icon to the selected hex
│   │               │       │   ├── The active_unit was adjacent to an enemy unit at the start of its move action ?
│   │               │       │   │   ├── YES → end_activation (ACTION, 1, FLED, MOVE, 1, 1)
│   │               │       │   │   └── NO → end_activation (ACTION, 1, MOVE, MOVE, 1, 1)
│   │               │       │   └── GO TO STEP : UNIT_ACTIVABLE_CHECK
│   │               │       ├── Left click on the active_unit → Move postponed
│   │               │       │   └── GO TO STEP : STEP : UNIT_ACTIVATION
│   │               │       ├── Right click on the active_unit → Move cancelled
│   │               │       │   ├── end_activation (NO, 0, PASS, MOVE, 1, 1)
│   │               │       │   └── GO TO STEP : UNIT_ACTIVABLE_CHECK
│   │               │       ├── Left click on another unit in activation pool → Move postponed
│   │               │       │   └── GO TO STEP : UNIT_ACTIVATION
│   │               │       └── Left OR Right click anywhere else on the board → Cancel Move hex selection
│   │               │           └── GO TO STEP : UNIT_ACTIVATION
│   │               └── NO → end_activation (NO, 0, PASS, MOVE, 1, 1)
│   ├── NO → If any, cancel the Highlight of the hexes in valid_move_destinations_pool
│   └── No more activable units → pass
└── End of MOVEMENT PHASE → Advance to shooting phase
```

Annexe 5 : Script BDD auth

```
-- ==============================================================================
-- Script de création et amorçage de la base d'authentification
-- Trazyn's Trials – config/users.db
-- Extrait équivalent à initialize_auth_db() dans services/api_server.py
-- ==============================================================================

-- Tables principales
CREATE TABLE IF NOT EXISTS profiles (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   code TEXT NOT NULL UNIQUE,
   label TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS users (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   login TEXT NOT NULL UNIQUE,
   password_hash TEXT NOT NULL,
   profile_id INTEGER NOT NULL REFERENCES profiles(id)
);

CREATE TABLE IF NOT EXISTS game_modes (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   code TEXT NOT NULL UNIQUE,
   label TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS options (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   code TEXT NOT NULL UNIQUE,
   label TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS profile_game_modes (
   profile_id INTEGER NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
   game_mode_id INTEGER NOT NULL REFERENCES game_modes(id) ON DELETE CASCADE,
   UNIQUE(profile_id, game_mode_id)
);

CREATE TABLE IF NOT EXISTS profile_options (
   profile_id INTEGER NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
   option_id INTEGER NOT NULL REFERENCES options(id) ON DELETE CASCADE,
   enabled INTEGER NOT NULL,
   UNIQUE(profile_id, option_id)
);

CREATE TABLE IF NOT EXISTS sessions (
   token TEXT PRIMARY KEY,
   user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
   created_at TEXT NOT NULL
);

-- Amorçage des profils
INSERT OR IGNORE INTO profiles (code, label) VALUES ('base', 'Joueur Base');
INSERT OR IGNORE INTO profiles (code, label) VALUES ('admin', 'Administrateur');

-- Amorçage des modes de jeu
INSERT OR IGNORE INTO game_modes (code, label) VALUES ('pve', 'Player vs Environment');
INSERT OR IGNORE INTO game_modes (code, label) VALUES ('pve_test', 'Player vs Environment Test');
INSERT OR IGNORE INTO game_modes (code, label) VALUES ('pvp', 'Player vs Player');
INSERT OR IGNORE INTO game_modes (code, label) VALUES ('pvp_test', 'Player vs Player Test');

-- Amorçage des options
INSERT OR IGNORE INTO options (code, label) VALUES ('show_advance_warning', 'Afficher avertissement mode advance');
INSERT OR IGNORE INTO options (code, label) VALUES ('auto_weapon_selection', 'Selection automatique d''arme');

-- Droits : profil base → pve, pve_test, pvp, pvp_test (via profile_game_modes)
-- Droits : profil admin → tous les modes (idem)
-- Options : profile_options lie chaque profil aux options avec enabled = 1
-- (Les INSERT exacts dépendent des id retournés par les SELECT ; en production
--  ils sont effectués par initialize_auth_db() en Python.)
```

Annexe 6 : Tableau des routes API

Tableau des routes API				
Méthode	**URL**	**Paramètres / corps**	**Codes retour**	**Remarques**
GET	/api/health	—	200	Healthcheck (Docker, monitoring).
POST	/api/auth/register	Body: { "login", "password" }	201, 400, 409	Création de compte (profil base).
POST	/api/auth/login	Body: { "login", "password" }	200, 400, 401	Retourne access_token, user, permissi ons.
GET	/api/auth/me	Header: Authorization: Bearer <token>	200,401	Session courante et permissions.
POST	/api/game/start	Header: Bearer. Body: { "pve_mode"?, "mode_code"?, "scenario_file"? }	200, 400, 401, 403, 500	Démarre une partie (mode selon droits).
POST	/api/game/action	Header: Bearer. Body: action sémantique (ex. move, shoot, wait)	200, 400, 401, 403, 500	Exécute une action joueur.
GET	/api/game/state	Header: Bearer	200, 400, 401	État courant de la partie.
POST	/api/game/reset	Header: Bearer	200, 400, 401	Réinitialise la partie.
POST	/api/game/ai-turn	Header: Bearer	200, 400, 401, 403, 500	Déclenche le tour de l'IA (PvE).
GET	/api/armies	—	200	Liste des armées (config).
GET	/api/config/board	—	200	Configuration plateau (scénario).
GET	/api/replay/list	Header: Bearer	200,401	Liste des fichiers replay.
GET	/api/replay/file/	Header: Bearer. Param: filename	200, 400, 401, 404	Contenu d'un replay (path traversal vérifié).
GET	/api/replay/default	Header: Bearer	200,401	Replay par défaut.
POST	/api/replay/parse	Header: Bearer. Body: contenu ou référence	200, 400, 401	Parse un replay.
GET	/api/debug/engine-test	Header: Bearer (admin)	200, 401, 403	Test moteur (debug).
GET	/api/debug/actions	Header: Bearer (admin)	200, 401, 403	Liste des actions possibles (debug).
GET	/	—	200	Servi par le frontend (Nginx en prod).

Routes protégées : toutes sauf /api/health, /api/auth/register, /api/auth/login, /api/armies, /api/config/board. Les modes pve_test et pvp_test sont réservés au profil admin.

Annexe 7 : Extraits de code

1. Point d'entrée step() – Moteur (délégation aux phases)

Fichier: engine/w40k_core.py
Contexte: Exécution d'une action gym (entier) : vérification des unités éligibles, conversion en action sémantique, délégation au handler de phase, mise à jour de l'observation.

```
# Lignes 962-976 (extrait)
# Normalize raw action once and keep it in game_state for deterministic
# policy-driven rule-choice resolution in gym training mode.
action_int = self.action_decoder.normalize_action_input(
    raw_action=action,
    phase=require_key(self.game_state, "phase"),
    source="w40k_core.step",
    action_space_size=len(action_mask),
)
self.game_state["_last_raw_action_int"] = action_int

# Convert gym integer action to semantic action (reuse precomputed mask+eligible_units)
semantic_action = self.action_decoder.convert_gym_action(
    action_int, self.game_state, action_mask=action_mask, eligible_units=eligible_units
)
self.game_state["_last_semantic_action"] = copy.deepcopy(semantic_action)
# ... puis _process_semantic_action(semantic_action) qui appelle le handler de phase
```

2. Début de phase mouvement – Pool d'activation

Fichier: engine/phase_handlers/movement_handlers.py
Contexte: Initialisation de la phase de mouvement : mise à jour de la phase, précalcul des hex adjacents aux ennemis, construction du pool d'unités éligibles au mouvement.

```
# Lignes 96-107 (extrait)
def movement_phase_start(game_state: Dict[str, Any]) -> Dict[str, Any]:
    """
    AI_MOVE.md: Initialize movement phase and build activation pool
    """
    # Set phase
    game_state["phase"] = "move"
    # ...
    units_cache = require_key(game_state, "units_cache")
    # Pre-compute enemy_adjacent_hexes once at phase start for all players present.
    players_present = set()
    for cache_entry in units_cache.values():
        player_raw = require_key(cache_entry, "player")
        # ...
        players_present.add(player_int)
    # Puis construction des pools (valid_move_destinations_pool, move_activation_pool, etc.)
```

3. Appel API frontend – Démarrage de partie

Fichier: frontend/src/hooks/useEngineAPI.ts
Contexte: Détection du mode depuis l'URL, récupération du token de session, envoi de la requête POST pour démarrer une partie.

```
// Lignes 301-314 (extrait)
const authSession = getAuthSession();
if (!authSession?.token) {
  throw new Error("Session utilisateur manquante. Merci de vous reconnecter.");
}

const response = await fetch(`${API_BASE}/game/start`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${authSession.token}`,
  },
  body: JSON.stringify(requestPayload),
});
```

Annexe 8 : step.log

```
=== STEP-BY-STEP ACTION LOG ===
AI_TURN.md COMPLIANCE: Actions that increment episode_steps are logged
INCREMENTED ACTIONS: move, shoot, charge, combat, wait (SUCCESS OR FAILURE)
NON-INCREMENTED: auto-skip ineligible units, phase transitions
FAILED ACTIONS: Still increment steps - unit consumed time/effort
================================================================================


[11:48:04] === EPISODE 1 START ===
[11:48:04] Scenario: scenario_training_bot-1
[11:48:04] Rosters: scale=100pts P1=p1_roster-03 (training/p1_roster-03.json) P2=p2_roster-01 (training/p2_roster-01.json)
[11:48:04] Opponent: RandomBot
[11:48:04] Walls: (2,5);(2,6);(2,7);(2,13);(2,14);(2,15);(3,4);(3,15);(4,4);(4,10);(4,16);(5,3);(5,10);(5,16);(6,3);(6,10);(6,17);(7,2);(7,10);(7,17);(8,2);(8,7);(8,8);(8,9);
(8,10);(8,14);(8,18);(9,1);(9,13);(9,18);(10,14);(11,13);(12,3);(12,4);(12,5);(12,6);(12,14);(12,15);(12,16);(12,17);(13,6);(14,6);(15,1);(15,6);(15,18);(16,2);(16,6);
(16,10);(16,11);(16,12);(16,13);(16,18);(17,2);(17,9);(17,17);(18,3);(18,10);(18,17);(19,3);(19,9);(19,16);(20,4);(20,10);(20,16);(21,4);(21,15);(22,5);(22,6);(22,7);
(22,13);(22,14);(22,15)
[11:48:04] Objectives: West:(2,10);(1,9);(2,9);(3,9);(3,10);(2,11);(1,10)|North:(12,1);(12,2);(11,1);(11,0);(12,0);(13,0);(13,1)|East:(22,10);(21,9);(22,9);(23,9);(23,10);
(22,11);(21,10)|South:(12,19);(11,18);(12,18);(13,18);(13,19);(12,20);(11,19)|Center:(12,10);(11,9);(12,9);(13,9);(13,10);(12,11);(11,10)
[11:48:04] Rules: {"primary_objective":[{"id":"objectives_control","name":"Objectives Control","identifier":"objectives_control","description":"Primary objective
scoring: from turn 2, score 5 VP for controlling >=1 objective, 5 VP for controlling >=2 objectives, and 5 VP for controlling more objectives than the opponent (max
15 per turn). Round 5: second player scores at end of fight phase instead of command phase.","scoring":{"start_turn":2,"max_points_per_turn":15,"rules":
[{"id":"control_at_least_one","points":5,"condition":"control_at_least_one"},{"id":"control_at_least_two","points":5,"condition":"control_at_least_two"},
{"id":"control_more_than_opponent","points":5,"condition":"control_more_than_opponent"}]},"timing":
{"default_phase":"command","round5_second_player_phase":"fight"},"control":
{"method":"oc_sum_greater","control_method":"sticky","tie_behavior":"no_control"}}]}
[11:48:04] Unit 1 (AssaultIntercessorJumpPackSergeantPowerFistHandFlamer) P1: Starting position (-1,-1), HP_MAX=2
[11:48:04] Unit 2 (AssaultIntercessorJumpPackSergeantPowerFistHandFlamer) P1: Starting position (-1,-1), HP_MAX=2
[11:48:04] Unit 3 (GreyHunterPackLeader) P1: Starting position (-1,-1), HP_MAX=2
[11:48:04] Unit 4 (GreyHunterPackLeader) P1: Starting position (-1,-1), HP_MAX=2
[11:48:04] Unit 5 (IntercessorSergeant) P1: Starting position (-1,-1), HP_MAX=2
[11:48:04] Unit 101 (TyranidWarriorRanged) P2: Starting position (-1,-1), HP_MAX=3
[11:48:04] Unit 102 (IntercessorGrenadeLauncher) P2: Starting position (-1,-1), HP_MAX=2
[11:48:04] Unit 103 (Intercessor) P2: Starting position (-1,-1), HP_MAX=2
[11:48:04] Unit 104 (Intercessor) P2: Starting position (-1,-1), HP_MAX=2
[11:48:04] Unit 105 (Intercessor) P2: Starting position (-1,-1), HP_MAX=2
[11:48:04] Unit 106 (Intercessor) P2: Starting position (-1,-1), HP_MAX=2
[11:48:04] === ACTIONS START ===
[11:48:04] E1 T1 P1 DEPLOYMENT : Unit 1(11,14) DEPLOYED from (-1,-1) to (11,14) [R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P2 DEPLOYMENT : Unit 101(12,2) DEPLOYED from (-1,-1) to (12,2) [R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P1 DEPLOYMENT : Unit 2(13,14) DEPLOYED from (-1,-1) to (13,14) [R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P2 DEPLOYMENT : Unit 102(2,4) DEPLOYED from (-1,-1) to (2,4) [R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P1 DEPLOYMENT : Unit 3(2,16) DEPLOYED from (-1,-1) to (2,16) [R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P2 DEPLOYMENT : Unit 103(22,4) DEPLOYED from (-1,-1) to (22,4) [R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P1 DEPLOYMENT : Unit 4(4,14) DEPLOYED from (-1,-1) to (4,14) [R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P2 DEPLOYMENT : Unit 104(11,1) DEPLOYED from (-1,-1) to (11,1) [R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P1 DEPLOYMENT : Unit 5(3,14) DEPLOYED from (-1,-1) to (3,14) [R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P2 DEPLOYMENT : Unit 105(2,3) DEPLOYED from (-1,-1) to (2,3) [R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P2 DEPLOYMENT : Unit 106(13,1) DEPLOYED from (-1,-1) to (13,1) [R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P1 MOVE : Unit 1(10,3) MOVED [FLY] from (11,14) to (10,3)[R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P1 MOVE : Unit 2(11,3) MOVED [FLY] from (13,14) to (11,3)[R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P1 MOVE : Unit 3(1,10) MOVED from (2,16) to (1,10)[R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P1 MOVE : Unit 4(3,8) MOVED from (4,14) to (3,8)[R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P1 MOVE : Unit 5(2,9) MOVED from (3,14) to (2,9)[R:+0.0] [SUCCESS]
[11:48:04] E1 T1 P1 SHOOT : Unit 1(10,3) SHOT Unit 106(13,1) with [Hand Flamer] - Hit 5(7+) [R:+0.6] [SUCCESS]
[11:48:04] E1 T1 P1 SHOOT : Unit 1(10,3) SHOT Unit 106(13,1) with [Hand Flamer] - Hit 4(7+) [R:+1.2] [SUCCESS]
[11:48:04] E1 T1 P1 SHOOT : Unit 2(11,3) SHOT Unit 104(11,1) with [Hand Flamer] - Hit 6(7+) [R:+0.3] [SUCCESS]
[11:48:04] E1 T1 P1 SHOOT : Unit 2(11,3) SHOT Unit 104(11,1) with [Hand Flamer] - Hit 6(7+) [R:+0.6] [SUCCESS]
[11:48:04] E1 T1 P1 SHOOT : Unit 2(11,3) SHOT Unit 104(11,1) with [Hand Flamer] - Hit 1(7+) [R:+0.9] [SUCCESS]
[11:48:04] E1 T1 P1 SHOOT : Unit 3(1,10) SHOT Unit 106(13,1) with [Boltgun] - Hit 2(3+) [R:+0.6] [SUCCESS]
[11:48:05] E1 T1 P1 SHOOT : Unit 3(1,10) SHOT Unit 106(13,1) with [Boltgun] - Hit 3(3+) - Wound 6(4+) - Save 2(3+) - Dmg:1HP [R:+1.2] [SUCCESS]
[11:48:05] E1 T1 P1 SHOOT : Unit 4(3,8) SHOT Unit 106(13,1) with [Boltgun] - Hit 4(3+) - Wound 5(4+) - Save 2(3+) - Dmg:1HP [R:+1.6] [SUCCESS]
[11:48:05] E1 T1 P1 SHOOT : Unit 4(3,8) SHOT Unit 104(11,1) with [Boltgun] - Hit 4(3+) - Wound 3(4+) [R:+2.2] [SUCCESS]
[11:48:05] E1 T1 P1 SHOOT : Unit 5(2,9) SHOT Unit 104(11,1) with [Plasma Pistol (Standard)] - Hit 1(3+) [R:+0.6] [SUCCESS]
[11:48:05] E1 T1 P1 CHARGE : Unit 1(11,2) CHARGED Unit 101(12,2) from (10,3) to (11,2) [Roll: 9] [R:+0.1] [SUCCESS]
[11:48:05] E1 T1 P1 CHARGE : Unit 1 IMPACTED [HAMMER OF WRATH] Unit 101(12,2) - Hit:4+:3(FAIL) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P1 CHARGE : Unit 2(12,1) CHARGED Unit 101(12,2) from (11,3) to (12,1) [Roll: 6] [R:+0.1] [SUCCESS]
[11:48:05] E1 T1 P1 CHARGE : Unit 2 IMPACTED [HAMMER OF WRATH] Unit 101(12,2) - Hit:4+:1(FAIL) [R:+0.0] [SUCCESS]
```

[11:48:05] E1 T1 P1 CHARGE : Unit 3(1,10) FAILED CHARGE to unit 102(2,4) [Roll: 2] [FAILED]
[11:48:05] E1 T1 P1 CHARGE : Unit 4(1,4) CHARGED Unit 102(2,4) from (3,8) to (1,4) [Roll: 6] [R:+0.1] [SUCCESS]
[11:48:05] E1 T1 P1 CHARGE : Unit 5(1,3) CHARGED Unit 102(2,4) from (2,9) to (1,3) [Roll: 7] [R:+0.1] [SUCCESS]
[11:48:05] E1 T1 P1 FIGHT : Unit 1(11,2) FOUGHT Unit 101(12,2) with [Power Fist] - Hit 6(3+) - Wound 3(3+) - Save 5(6+) - Dmg:2HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P1 FIGHT : Unit 1(11,2) FOUGHT Unit 101(12,2) with [Power Fist] - Hit 1(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P1 FIGHT : Unit 1(11,2) FOUGHT Unit 101(12,2) with [Power Fist] - Hit 1(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P1 FIGHT : Unit 2(12,1) FOUGHT Unit 101(12,2) with [Power Fist] - Hit 1(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P1 FIGHT : Unit 2(12,1) FOUGHT Unit 101(12,2) with [Power Fist] - Hit 4(3+) - Wound 1(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P1 FIGHT : Unit 2(12,1) FOUGHT Unit 101(12,2) with [Power Fist] - Hit 1(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P1 FIGHT : Unit 4(1,4) FOUGHT Unit 102(2,4) with [Power Fist] - Hit 6(3+) - Wound 2(2+) - Save 2(5+) - Dmg:2HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P1 FIGHT : Unit 5(1,3) FOUGHT Unit 105(2,3) with [Power Fist] - Hit 1(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P1 FIGHT : Unit 5(1,3) FOUGHT Unit 105(2,3) with [Power Fist] - Hit 5(3+) - Wound 1(2+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P1 FIGHT : Unit 5(1,3) FOUGHT Unit 105(2,3) with [Power Fist] - Hit 5(3+) - Wound 4(2+) - Save 6(5+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 101(12,2) FOUGHT Unit 1(11,2) with [Rending Claws] - Hit 1(2+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 101(12,2) FOUGHT Unit 1(11,2) with [Rending Claws] - Hit 1(2+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 101(12,2) FOUGHT Unit 1(11,2) with [Rending Claws] - Hit 1(2+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 101(12,2) FOUGHT Unit 1(11,2) with [Rending Claws] - Hit 3(2+) - Wound 2(4+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 104(11,1) FOUGHT Unit 1(11,2) with [Close Combat Weapon] - Hit 6(3+) - Wound 1(4+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 104(11,1) FOUGHT Unit 1(11,2) with [Close Combat Weapon] - Hit 6(3+) - Wound 3(4+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 104(11,1) FOUGHT Unit 1(11,2) with [Close Combat Weapon] - Hit 5(3+) - Wound 3(4+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 105(2,3) FOUGHT Unit 5(1,3) with [Close Combat Weapon] - Hit 3(3+) - Wound 4(4+) - Save 5(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 105(2,3) FOUGHT Unit 5(1,3) with [Close Combat Weapon] - Hit 4(3+) - Wound 4(4+) - Save 3(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 105(2,3) FOUGHT Unit 5(1,3) with [Close Combat Weapon] - Hit 6(3+) - Wound 3(4+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 MOVE : Unit 101(18,5) FLED from (12,2) to (18,5) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 MOVE : Unit 103(24,2) MOVED from (22,4) to (24,2)[R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 MOVE : Unit 104(10,1) FLED from (11,1) to (10,1) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 MOVE : Unit 105(4,1) FLED from (2,3) to (4,1) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 SHOOT : Unit 101(18,5) SHOT Unit 2(12,1) with [Deathspitter] - Hit 4(4+) - Wound 3(3+) - Save 2(4+) - Dmg:1HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 SHOOT : Unit 101(18,5) SHOT Unit 2(12,1) with [Deathspitter] - Hit 4(4+) - Wound 6(3+) - Save 6(4+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 SHOOT : Unit 101(18,5) SHOT Unit 2(12,1) with [Deathspitter] - Hit 3(4+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 SHOOT : Unit 103(21,3) ADVANCED from (24,2) to (21,3) [Roll: 3] [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 CHARGE : Unit 101(12,2) CHARGED [ADAPTABLE PREDATORS] Unit 1(11,2) from (18,5) to (12,2) [Roll: 8] [R:+0.1] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 101(12,2) FOUGHT Unit 1(11,2) with [Rending Claws] - Hit 2(2+) - Wound 5(4+) - Save 2(5+) - Dmg:1HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 101(12,2) FOUGHT Unit 1(11,2) with [Rending Claws] - Hit 2(2+) - Wound 5(4+) - Save 4(5+) - Dmg:1HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 101(12,2) FOUGHT Unit 2(12,1) with [Rending Claws] - Hit 3(2+) - Wound 3(4+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P2 FIGHT : Unit 101(12,2) FOUGHT Unit 2(12,1) with [Rending Claws] - Hit 5(2+) - Wound 4(4+) - Save 5(5+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T1 P1 FIGHT : Unit 2(12,1) FOUGHT Unit 101(12,2) with [Power Fist] - Hit 3(3+) - Wound 6(3+) - Save 1(6+) - Dmg:2HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P1 MOVE : Unit 2(2,0) MOVED [FLY] from (12,1) to (2,0)[R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P1 MOVE : Unit 3(3,5) MOVED from (1,10) to (3,5)[R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P1 MOVE : Unit 4(2,2) MOVED from (1,4) to (2,2)[R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P1 MOVE : Unit 5(3,2) MOVED from (1,3) to (3,2)[R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P1 SHOOT : Unit 2(2,0) SHOT Unit 104(10,1) with [Hand Flamer] - Hit 5(7+) [R:+0.6] [SUCCESS]
[11:48:05] E1 T2 P1 SHOOT : Unit 2(2,0) SHOT Unit 104(10,1) with [Hand Flamer] - Hit 1(7+) [R:+1.2] [SUCCESS]
[11:48:05] E1 T2 P1 SHOOT : Unit 2(2,0) SHOT Unit 104(10,1) with [Hand Flamer] - Hit 6(7+) [R:+1.8] [SUCCESS]
[11:48:05] E1 T2 P1 SHOOT : Unit 2(2,0) SHOT Unit 104(10,1) with [Hand Flamer] - Hit 5(7+) [R:+2.4] [SUCCESS]
[11:48:05] E1 T2 P1 SHOOT : Unit 2(2,0) SHOT Unit 104(10,1) with [Hand Flamer] - Hit 6(7+) [R:+3.0] [SUCCESS]
[11:48:05] E1 T2 P1 SHOOT : Unit 3(4,5) ADVANCED from (3,5) to (4,5) [Roll: 2] [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P1 SHOOT : Unit 4(2,2) SHOT Unit 105(4,1) with [Boltgun] - Hit 2(3+) [R:+0.3] [SUCCESS]
[11:48:05] E1 T2 P1 SHOOT : Unit 4(2,2) SHOT Unit 105(4,1) with [Boltgun] - Hit 4(3+) - Wound 1(4+) [R:+0.6] [SUCCESS]
[11:48:05] E1 T2 P1 SHOOT : Unit 5(3,2) SHOT Unit 105(4,1) with [Plasma Pistol (Standard)] - Hit 1(3+) [R:+0.3] [SUCCESS]
[11:48:05] E1 T2 P1 CHARGE : Unit 2(9,0) CHARGED Unit 104(10,1) from (2,0) to (9,0) [Roll: 7] [R:+0.1] [SUCCESS]
[11:48:05] E1 T2 P1 CHARGE : Unit 2 IMPACTED [HAMMER OF WRATH] Unit 104(10,1) - Hit:4+:2(FAIL) [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P1 CHARGE : Unit 4(3,1) CHARGED Unit 105(4,1) from (2,2) to (3,1) [Roll: 5] [R:+0.1] [SUCCESS]
[11:48:05] E1 T2 P1 CHARGE : Unit 5(4,2) CHARGED Unit 105(4,1) from (3,2) to (4,2) [Roll: 7] [R:+0.1] [SUCCESS]
[11:48:05] E1 T2 P1 FIGHT : Unit 2(9,0) FOUGHT Unit 104(10,1) with [Power Fist] - Hit 2(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P1 FIGHT : Unit 2(9,0) FOUGHT Unit 104(10,1) with [Power Fist] - Hit 4(3+) - Wound 2(2+) - Save 6(5+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P1 FIGHT : Unit 2(9,0) FOUGHT Unit 104(10,1) with [Power Fist] - Hit 2(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P1 FIGHT : Unit 4(3,1) FOUGHT Unit 105(4,1) with [Power Fist] - Hit 6(3+) - Wound 3(2+) - Save 3(5+) - Dmg:2HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P1 FIGHT : Unit 5 WAIT [SUCCESS]
[11:48:05] E1 T2 P2 FIGHT : Unit 104(10,1) FOUGHT Unit 2(9,0) with [Close Combat Weapon] - Hit 2(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P2 FIGHT : Unit 104(10,1) FOUGHT Unit 2(9,0) with [Close Combat Weapon] - Hit 5(3+) - Wound 4(4+) - Save 4(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P2 FIGHT : Unit 104(10,1) FOUGHT Unit 2(9,0) with [Close Combat Weapon] - Hit 4(3+) - Wound 5(4+) - Save 1(3+) - Dmg:1HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P2 MOVE : Unit 103(23,3) MOVED from (21,3) to (23,3)[R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P2 MOVE : Unit 104(6,2) MOVED from (10,1) to (6,2)[R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P2 SHOOT : Unit 103(18,2) ADVANCED from (23,3) to (18,2) [Roll: 5] [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P2 SHOOT : Unit 104(6,2) SHOT Unit 4(3,1) with [Bolt Rifle] - Hit 3(3+) - Wound 5(4+) - Save 3(4+) - Dmg:1HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P2 SHOOT : Unit 104(6,2) SHOT Unit 4(3,1) with [Bolt Rifle] - Hit 3(3+) - Wound 5(4+) - Save 6(4+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P2 SHOOT : Unit 104(6,2) SHOT Unit 5(4,2) with [Bolt Rifle] - Hit 3(3+) - Wound 2(4+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T2 P2 CHARGE : Unit 104(6,2) FAILED CHARGE to unit 3(4,5) [Roll: 6] [FAILED]
[11:48:05] E1 T3 P1 MOVE : Unit 3(6,4) MOVED from (4,5) to (6,4)[R:+0.0] [SUCCESS]
[11:48:05] E1 T3 P1 MOVE : Unit 4(4,1) MOVED from (3,1) to (4,1)[R:+0.0] [SUCCESS]
[11:48:05] E1 T3 P1 MOVE : Unit 5(4,3) MOVED from (4,2) to (4,3)[R:+0.0] [SUCCESS]

```
[11:48:05] E1 T3 P1 SHOOT : Unit 3(7,3) ADVANCED from (6,4) to (7,3) [Roll: 5] [R:+0.0] [SUCCESS]
[11:48:05] E1 T3 P1 SHOOT : Unit 4(4,1) SHOT Unit 104(6,2) with [Boltgun] - Hit 4(3+) - Wound 4(4+) - Save 4(3+) [R:+0.3] [SUCCESS]
[11:48:05] E1 T3 P1 SHOOT : Unit 4(4,1) SHOT Unit 104(6,2) with [Boltgun] - Hit 5(3+) - Wound 3(4+) [R:+0.6] [SUCCESS]
[11:48:05] E1 T3 P1 SHOOT : Unit 5(4,3) SHOT Unit 104(6,2) with [Plasma Pistol (Standard)] - Hit 1(3+) [R:+0.3] [SUCCESS]
[11:48:05] E1 T3 P1 CHARGE : Unit 4(5,1) CHARGED Unit 104(6,2) from (4,1) to (5,1) [Roll: 5] [R:+0.1] [SUCCESS]
[11:48:05] E1 T3 P1 CHARGE : Unit 5(5,2) CHARGED Unit 104(6,2) from (4,3) to (5,2) [Roll: 6] [R:+0.1] [SUCCESS]
[11:48:05] E1 T3 P1 FIGHT : Unit 4(5,1) FOUGHT Unit 104(6,2) with [Power Fist] - Hit 5(3+) - Wound 1(2+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T3 P1 FIGHT : Unit 4(5,1) FOUGHT Unit 104(6,2) with [Power Fist] - Hit 1(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T3 P1 FIGHT : Unit 5(5,2) FOUGHT Unit 104(6,2) with [Power Fist] - Hit 4(3+) - Wound 4(2+) - Save 1(5+) - Dmg:2HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T3 P2 MOVE : Unit 103(24,0) MOVED from (18,2) to (24,0)[R:+0.0] [SUCCESS]
[11:48:05] E1 T3 P2 SHOOT : Unit 103(19,2) ADVANCED from (24,0) to (19,2) [Roll: 5] [R:+0.0] [SUCCESS]
[11:48:05] E1 T4 P1 MOVE : Unit 3(13,0) MOVED from (7,3) to (13,0)[R:+0.0] [SUCCESS]
[11:48:05] E1 T4 P1 MOVE : Unit 4(11,0) MOVED from (5,1) to (11,0)[R:+0.0] [SUCCESS]
[11:48:05] E1 T4 P1 MOVE : Unit 5(11,1) MOVED from (5,2) to (11,1)[R:+0.0] [SUCCESS]
[11:48:05] E1 T4 P1 SHOOT : Unit 3(15,0) ADVANCED from (13,0) to (15,0) [Roll: 2] [R:+0.0] [SUCCESS]
[11:48:05] E1 T4 P1 SHOOT : Unit 3(15,0) SHOT Unit 103(19,2) with [Boltgun] - Hit 2(3+) [R:+0.3] [SUCCESS]
[11:48:05] E1 T4 P1 SHOOT : Unit 3(15,0) SHOT Unit 103(19,2) with [Boltgun] - Hit 1(3+) [R:+0.6] [SUCCESS]
[11:48:05] E1 T4 P1 SHOOT : Unit 4(12,0) ADVANCED from (11,0) to (12,0) [Roll: 1] [R:+0.0] [SUCCESS]
[11:48:05] E1 T4 P1 SHOOT : Unit 5(14,0) ADVANCED from (11,1) to (14,0) [Roll: 3] [R:+0.0] [SUCCESS]
[11:48:05] E1 T4 P2 MOVE : Unit 103(24,6) MOVED from (19,2) to (24,6)[R:+0.0] [SUCCESS]
[11:48:05] E1 T4 P2 SHOOT : Unit 103(20,3) ADVANCED from (24,6) to (20,3) [Roll: 5] [R:+0.0] [SUCCESS]
[11:48:05] E1 T4 P2 SHOOT : Unit 103(20,3) SHOT Unit 5(14,0) with [Bolt Rifle] - Hit 4(3+) - Wound 5(4+) - Save 1(4+) - Dmg:1HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T4 P2 SHOOT : Unit 103(20,3) SHOT Unit 5(14,0) with [Bolt Rifle] - Hit 1(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T4 P2 SHOOT : Unit 103(20,3) SHOT Unit 3(15,0) with [Bolt Rifle] - Hit 2(3+) [R:+0.0] [SUCCESS]
[11:48:05] E1 T5 P1 MOVE : Unit 3(18,2) MOVED from (15,0) to (18,2)[R:+0.0] [SUCCESS]
[11:48:05] E1 T5 P1 MOVE : Unit 4(17,1) MOVED from (12,0) to (17,1)[R:+0.0] [SUCCESS]
[11:48:05] E1 T5 P1 MOVE : Unit 5(8,3) MOVED from (14,0) to (8,3)[R:+0.0] [SUCCESS]
[11:48:05] E1 T5 P1 SHOOT : Unit 3(18,2) SHOT Unit 103(20,3) with [Boltgun] - Hit 3(3+) - Wound 1(4+) [R:+0.3] [SUCCESS]
[11:48:05] E1 T5 P1 SHOOT : Unit 3(18,2) SHOT Unit 103(20,3) with [Boltgun] - Hit 3(3+) - Wound 5(4+) - Save 3(3+) [R:+0.6] [SUCCESS]
[11:48:05] E1 T5 P1 SHOOT : Unit 4(17,1) SHOT Unit 103(20,3) with [Boltgun] - Hit 2(3+) [R:+0.3] [SUCCESS]
[11:48:05] E1 T5 P1 SHOOT : Unit 4(17,1) SHOT Unit 103(20,3) with [Boltgun] - Hit 3(3+) - Wound 4(4+) - Save 6(3+) [R:+0.6] [SUCCESS]
[11:48:05] E1 T5 P1 SHOOT : Unit 5 WAIT [SUCCESS]
[11:48:05] E1 T5 P1 CHARGE : Unit 3(19,2) CHARGED Unit 103(20,3) from (18,2) to (19,2) [Roll: 10] [R:+0.1] [SUCCESS]
[11:48:05] E1 T5 P1 CHARGE : Unit 4(20,2) CHARGED Unit 103(20,3) from (17,1) to (20,2) [Roll: 5] [R:+0.1] [SUCCESS]
[11:48:05] E1 T5 P1 CHARGE : Unit 5 WAIT [SUCCESS]
[11:48:05] E1 T5 P1 FIGHT : Unit 3(19,2) FOUGHT Unit 103(20,3) with [Power Fist] - Hit 6(3+) - Wound 4(2+) - Save 4(5+) - Dmg:2HP [R:+0.0] [SUCCESS]
[11:48:05] E1 T5 P1 FIGHT : Unit 4 WAIT [SUCCESS]
[11:48:05] EPISODE END: Winner=1, Method=objectives, Actions=139, Steps=136, Total=235, Duration=0.898s
[11:48:05] OBJECTIVE CONTROL: Obj1:P1_OC=0,P2_OC=0,Ctrl=1 | Obj2:P1_OC=0,P2_OC=0,Ctrl=1 | Obj3:P1_OC=0,P2_OC=0,Ctrl=None |
Obj4:P1_OC=0,P2_OC=0,Ctrl=None | Obj5:P1_OC=0,P2_OC=0,Ctrl=None
================================================================================
```

Annexe 9 : output de Analyzer.py

Analyzer.py's Game Analysis : 1/2

```
Analyzing step.log...
Généré le: 2026-03-08 01:15:45
================================================================================
================================================================================
STEP.LOG ANALYSIS - GAME RULES VALIDATION
================================================================================

================================================================================
GAME ANALYSIS
================================================================================

Total Episodes: 300
Total Actions: 44199
Episode Actions: 147.3 (average)
  Min: 63 (Episode 25) - (duration: 0.46s)
  Max: 300 (Episode 152) - (duration: 1.50s)
Episode Durations: 0.77s (average)
  Min: 0.36s (Episode 286) - (actions: 72)
  Max: 1.54s (Episode 253) - (actions: 270)

================================================================================
📊 BOT EVALUATION RESULTS
================================================================================
--------------------------------------------------------------------------------
WIN METHODS          Agent Wins (P1)      Bot Wins (P2)
--------------------------------------------------------------------------------
Elimination               0 (  0.0%)        0 (  0.0%)
Objectives              153 ( 90.5%)      105 ( 80.8%)
Value Tiebreaker         16 (  9.5%)       25 ( 19.2%)
--------------------------------------------------------------------------------
TOTAL WINS              169 ( 56.3%)      130 ( 43.3%)
DRAWS                     1 (  0.3%)

--------------------------------------------------------------------------------
VICTORY POINTS (OBJECTIVES)
--------------------------------------------------------------------------------
Player          Min      Avg      Max
P1             0.00    19.57    60.00
P2             0.00    28.72    60.00
--------------------------------------------------------------------------------
WINS BY SCENARIO                          Agent (P1)      Bot (P2)        Draws
--------------------------------------------------------------------------------
bot-1                                     15 (50.0%)    14 (46.7%)     1 ( 3.3%)
bot-10                                    21 (70.0%)     9 (30.0%)     0 ( 0.0%)
bot-2                                     21 (70.0%)     9 (30.0%)     0 ( 0.0%)
bot-3                                     14 (46.7%)    16 (53.3%)     0 ( 0.0%)
bot-4                                     15 (50.0%)    15 (50.0%)     0 ( 0.0%)
bot-5                                     13 (43.3%)    17 (56.7%)     0 ( 0.0%)
bot-6                                     22 (73.3%)     8 (26.7%)     0 ( 0.0%)
bot-7                                     18 (60.0%)    12 (40.0%)     0 ( 0.0%)
bot-8                                     16 (53.3%)    14 (46.7%)     0 ( 0.0%)
bot-9                                     14 (46.7%)    16 (53.3%)     0 ( 0.0%)
```

Analyzer.py's Game Analysis : 2/2

```
  --------------------------------------------------------------------------------
  TURN DISTRIBUTION
  --------------------------------------------------------------------------------
  Turn 5: 300 games (100.0%)
  --------------------------------------------------------------------------------
  ACTIONS BY TYPE   Agent (P1)          Bot (P2)
  --------------------------------------------------------------------------------
  shoot          6101 ( 32.5%)     5205 ( 20.9%)
  move           5001 ( 26.6%)     5789 ( 23.3%)
  fight          2692 ( 14.3%)     4459 ( 17.9%)
  wait           3632 ( 19.3%)     3398 ( 13.7%)
  advance         991 (  5.3%)     2730 ( 11.0%)
  charge          177 (  0.9%)     1424 (  5.7%)
  other            61 (  0.3%)     1088 (  4.4%)
  skip            141 (  0.8%)      784 (  3.2%)
  --------------------------------------------------------------------------------
  SHOOTING BEHAVIOR Agent (P1)          Bot (P2)
  --------------------------------------------------------------------------------
  Shoot          6101 ( 56.2%)     5205 ( 43.0%)
  Skip            141 (  1.3%)      784 (  6.5%)
  Advance         991 (  9.1%)     2730 ( 22.5%)
  Wait (targets)      0 (  0.0%)        0 (  0.0%)
  Wait (no targets)     61 (  0.6%)     1446 ( 11.9%)
  Shoot+Advance    215 (  2.0%)      198 (  1.6%)
  --------------------------------------------------------------------------------
  PISTOL WEAPON SHOTS BY ADJACENCY    Agent (P1)         Bot (P2)
  --------------------------------------------------------------------------------
  PISTOL shots (adjacent):           40 ( 44.0%)       5 (  0.9%)
  PISTOL shots (not adjacent):       51 ( 56.0%)     574 ( 99.1%)
  Total PISTOL shots:                91              579
  Non-PISTOL shots (adjacent):        0                0
  --------------------------------------------------------------------------------
  SHOOTING VALIDITY                   Agent (P1)         Bot (P2)
  --------------------------------------------------------------------------------
  Invalid shots total:                 0 (  0.0%)       0 (  0.0%)
  No LoS:                              0                0
  Out of range:                        0                0
  Adjacent non-pistol:                 0                0

  --------------------------------------------------------------------------------
  WAIT BEHAVIOR BY PHASE     Agent (P1)         Bot (P2)
  --------------------------------------------------------------------------------
  MOVE phase waits:                  0                0
  SHOOT waits (enemies in LOS):      0                0
  SHOOT waits (no LOS):             61             1446

  --------------------------------------------------------------------------------
  TARGET PRIORITY ANALYSIS           Agent (P1)         Bot (P2)
  --------------------------------------------------------------------------------
  FAILURES (shot full HP while
    wounded in LOS):               491 (  8.0%)    1226 ( 23.6%)
  SUCCESS (shot wounded or
    no wounded in LOS):           5610 ( 92.0%)    3979 ( 76.4%)
  Total shots:                    6101             5205

  --------------------------------------------------------------------------------
  ENEMY DEATH ORDER ANALYSIS
  --------------------------------------------------------------------------------
  Total episodes with kills: 300

  Most common death orders:
    IntercessorGrenadeLauncher(1) -> IntercessorGrenadeLauncher(2) -> IntercessorGrenadeLauncher(3) -> GreyHunter(4) -> GreyHunter(5): 3 times (1.0%)
    IntercessorGrenadeLauncher(1) -> IntercessorGrenadeLauncher(2) -> IntercessorGrenadeLauncher(3): 2 times (0.7%)
    IntercessorGrenadeLauncher(1) -> Intercessor(105) -> IntercessorGrenadeLauncher(2) -> IntercessorGrenadeLauncher(3) -> GreyHunter(4) -> GreyHunte
  r(5): 2 times (0.7%)
    IntercessorGrenadeLauncher(1) -> IntercessorGrenadeLauncher(2) -> IntercessorGrenadeLauncher(3) -> GreyHunter(5) -> GreyHunter(4): 2 times (0.7%)
    IntercessorGrenadeLauncher(1) -> IntercessorGrenadeLauncher(2) -> GreyHunter(4) -> GreyHunter(5) -> IntercessorGrenadeLauncher(3): 2 times (0.7%)
    TyranidWarriorRanged(101) -> IntercessorGrenadeLauncher(1) -> IntercessorGrenadeLauncher(2) -> IntercessorGrenadeLauncher(3) -> GreyHunter(4) ->
  GreyHunter(5): 2 times (0.7%)
    AssaultIntercessorSergeant(101) -> AssaultIntercessor(103) -> AssaultIntercessor(105) -> AssaultIntercessor(104): 2 times (0.7%)
    Intercessor(104) -> Intercessor(105) -> IntercessorGrenadeLauncher(1) -> IntercessorGrenadeLauncher(2) -> IntercessorGrenadeLauncher(3) -> Tyrani
  dWarriorRanged(101): 1 times (0.3%)
    Intercessor(104) -> IntercessorGrenadeLauncher(102) -> TyranidWarriorRanged(101) -> Intercessor(105) -> IntercessorGrenadeLauncher(1) -> Interces
  sor(106) -> IntercessorGrenadeLauncher(2) -> Intercessor(103): 1 times (0.3%)
    IntercessorGrenadeLauncher(1) -> Intercessor(106) -> Intercessor(103) -> Intercessor(105) -> IntercessorGrenadeLauncher(102) -> IntercessorGrenad
  eLauncher(2) -> GreyHunter(4): 1 times (0.3%)

Kills by player:
  Agent (P1) kills: 1509
  Bot (P2) kills:   841
```

Analyzer.py's Bug Analysis : 1/3

```
================================================================================
DEBUGGING
================================================================================
Sections:
  1.1 MOVEMENT ERRORS
  1.2 SHOOTING ERRORS
  1.3 CHARGE ERRORS
  1.4 FIGHT ERRORS
  1.5 ACTION PHASE ACCURACY
  1.6 DOUBLE-ACTIVATION PAR PHASE
  1.7 SPECIAL RULES USAGE
  1.8 WEAPONS RULES USAGE
  2.1 DEAD UNITS INTERACTIONS
  2.2 POSITION / LOG COHERENCE
  2.3 DMG ISSUES
  2.4 EPISODES STATISTICS
  2.5 EPISODES ENDING
  2.6 SAMPLE MISSING
  2.7 CORE ISSUES

--------------------------------------------------------------------------------
1.1 MOVEMENT ERRORS                 Agent (P1)        Bot (P2)
--------------------------------------------------------------------------------
Moves into walls:                  0                 0
Moves to adjacent enemy:           0                 0
Move with adjacent_before:         0                 0
Move distance > MOVE:              0                 0
Move path blocked (BFS):           0                 0
Reactive moves applied:            0               233
Reactive moves declined:           0                 0
Reactive moves abnormal:           0                 0
Reactive to adjacent enemy:        0                 0
Reactive into wall:                0                 0
Reactive path blocked (BFS):       0                 0
Reactive distance > roll:          0                 0

--------------------------------------------------------------------------------
1.2 SHOOTING ERRORS                      Agent (P1)        Bot (P2)
--------------------------------------------------------------------------------
Tirs invalides
(LoS/portée/adjacent non-pistol):             0                 0
Shots over RNG_NB:                                     0                 0
COMBI profiles in same phase:                          0                 0
Shoot through wall:                0                 0
Shoot after flee:                      0                 0
Shoot after flee (rule):         267               117
Shoot at friendly unit:                0                 0
Shoot at engaged enemy:                0                 0
Non-pistol adjacent shots:             0                 0
Advance after shoot:                   0                 0
Advance twice in SHOOT:                0                 0
Advance distance > roll:               0                 0
Advances from adjacent hex:            0                 0
Advance path blocked (BFS):            0                 0

--------------------------------------------------------------------------------
1.3 CHARGE ERRORS                   Agent (P1)        Bot (P2)
--------------------------------------------------------------------------------
Charges from adjacent hex:         0                 0
Charges after flee:                0                 0
Charge after flee (rule):          0                62
Charge after advance (rule):       0               486
Charges after advance:             0                 0
Distance > roll:                   0                 0

--------------------------------------------------------------------------------
1.4 FIGHT ERRORS                    Agent (P1)        Bot (P2)
--------------------------------------------------------------------------------
Fight from non-adjacent hex:       0                 0
Fight a friendly unit:             0                 0
Attacks over CC_NB:                0                 0
Fight alternation violations:      0                 0
```

Analyzer.py's Bug Analysis : 2/3

```
--------------------------------------------------------------------------------
1.5 ACTION PHASE ACCURACY
--------------------------------------------------------------------------------
Action          Total    Wrong   Accuracy
--------------------------------------------------------------------------------
MOVE            10790        0     100.0%
FLED              526        0     100.0%
SHOOT           11306        0     100.0%
ADVANCE          3721        0     100.0%
CHARGE           1601        0     100.0%
FIGHT            7151        0     100.0%

--------------------------------------------------------------------------------
1.6 DOUBLE-ACTIVATION PAR PHASE
--------------------------------------------------------------------------------
No double-activation detected.

--------------------------------------------------------------------------------
1.7 SPECIAL RULES USAGE                  Unit                                                    P1         P2   Validité
--------------------------------------------------------------------------------
adaptable_predators                      TyranidWarriorRanged                                     0          0         OK
adrenalised_onslaught                    TyranidWarriorMelee                                      0          0         OK
charge_after_advance                     BloodClaw                                                0          0         OK
charge_after_advance                     BloodClawPackLeader                                      0          0         OK
charge_after_advance                     Hormagaunt                                               0        486         OK
charge_after_flee                        TyranidWarriorRanged                                     0         62         OK
charge_impact                            AssaultIntercessorJumpPack                               0          0         OK
charge_impact                            AssaultIntercessorJumpPackPlasmaPistol                   0          0         OK
charge_impact                            AssaultIntercessorJumpPackSergeantPowerFistHandFlamer    0          0         OK
closest_target_penetration               AggressorBoltStorm                                       0          0         OK
closest_target_penetration               AggressorFlamestorm                                      0          0         OK
cunning_hunters                          GreyHunter                                               0          0         OK
cunning_hunters                          GreyHunterPackLeader                                     0          0         OK
cunning_hunters                          GreyHunterPlasmaGun                                      0          0         OK
move_after_shouting                      Gargoyle                                                 0          0         OK
reactive_move                            FenrisianWolf                                            0          0         OK
reactive_move                            Termagant                                                0        233         OK
reroll_1_save_fight                      TyranidWarriorMelee                                      0          0         OK
reroll_1_tohit_fight                     TyranidWarriorMelee                                      0          0         OK
reroll_1_towound                         AssaultIntercessor                                       0          0         OK
reroll_1_towound                         AssaultIntercessorSergeant                               0          0         OK
reroll_towound_target_on_objective       AssaultIntercessor                                       0          0         OK
reroll_towound_target_on_objective       AssaultIntercessorSergeant                               0          0         OK
shoot_after_advance                      GreyHunter                                             215          0         OK
shoot_after_advance                      GreyHunterPackLeader                                     0          0         OK
shoot_after_advance                      GreyHunterPlasmaGun                                      0          0         OK
shoot_after_flee                         GreyHunter                                             267          0         OK
shoot_after_flee                         GreyHunterPackLeader                                     0          0         OK
shoot_after_flee                         GreyHunterPlasmaGun                                      0          0         OK
shoot_after_flee                         TyranidWarriorRanged                                     0        117         OK
targeted_intercession                    AssaultIntercessor                                       0          0         OK
targeted_intercession                    AssaultIntercessorSergeant                               0          0         OK

  Rule-choice compliance (selected option vs used option)
  Rule                                Unit                                P1 OK    P2 OK  P1 MISS  P2 MISS   P1 BAD   P2 BAD
  No rule-choice usage recorded.
```


Holberton
actual digital school

Analyzer.py's Bug Analysis : 3/3

```
--------------------------------------------------------------------------------
1.8 WEAPONS RULES USAGE      Weapon                                          P1        P2   Validité
--------------------------------------------------------------------------------
Assault                      Bolt Rifle (Intercessor)                         0       133        OK
Assault                      Fleshborer (Termagant)                           0        16        OK
Assault                      Heavy Bolt Rifle (HeavyIntercessor)              0        49        OK
Heavy                        Bolt Rifle (Intercessor)                         0        78        OK
Heavy                        Heavy Bolt Rifle (HeavyIntercessor)              0        45        OK
Pistol                       Bolt Pistol (GreyHunter)                        11         0        OK
Pistol                       Bolt Pistol (HeavyIntercessor)                   0         4        OK
Pistol                       Bolt Pistol (Intercessor)                        0         1        OK
Pistol                       Bolt Pistol (IntercessorGrenadeLauncher)        29         0        OK
Devastating_wounds           GLOBAL (correct)                                 0        40        OK
Rapid_fire                   GLOBAL (correct)                                 0         0        OK

--------------------------------------------------------------------------------
2.1 DEAD UNITS INTERACTIONS         Agent (P1)        Bot (P2)
--------------------------------------------------------------------------------
Incomplete episodes:                0                 0
Dead unit moving:                   0                 0
Dead unit shooting:                 0                 0
Shoot at dead unit:                 0                 0
Dead unit advancing:                0                 0
Dead unit charging:                 0                 0
Charge a dead unit:                 0                 0
Dead unit fighting:                 0                 0
Fight a dead unit:                  0                 0
Dead unit waiting:                  0                 0
Dead unit skipping:                 0                 0
Unités revenues après mort:         0                 0

--------------------------------------------------------------------------------
2.2 POSITION / LOG COHERENCE
--------------------------------------------------------------------------------
MOVE     total= 10790 mismatch=     0 missing=     0 mismatch_pct=  0.00%
ADVANCE  total=  3721 mismatch=     0 missing=     0 mismatch_pct=  0.00%
CHARGE   total=  1601 mismatch=     0 missing=     0 mismatch_pct=  0.00%
---
Total collisions (2+ units in same hex): 0

--------------------------------------------------------------------------------
2.3 DMG ISSUES                      Agent (P1)        Bot (P2)
--------------------------------------------------------------------------------
Missing unit_hp on damage:        0                 0
Dmg > HP_CUR (overkill):          0                 0

--------------------------------------------------------------------------------
2.4 EPISODES STATISTICS
--------------------------------------------------------------------------------
Longest episode (average duration): Episode 253 - 1.54s (avg 0.77s)
Episode with most actions (average action number): Episode 152 - 300 actions (avg 147.3)

--------------------------------------------------------------------------------
2.5 EPISODES ENDING                 Agent (P1)        Bot (P2)
--------------------------------------------------------------------------------
Incomplete episodes:              0                 0
Episodes without win_method:      0                 0

--------------------------------------------------------------------------------
2.6 SAMPLE MISSING
--------------------------------------------------------------------------------
MOVE    --- [01:11:54] E1 T1 P1 MOVE : Unit 1(1,10) MOVED from (2,16) to (1,10)[R:+0.0] [SUCCESS]
SHOOT   --- [01:11:54] E1 T1 P1 SHOOT : Unit 1(1,10) SHOT Unit 104(3,6) with [Astartes Grenade Launcher (Krak)] - Hit 3(3+) - Wound 4(2+) - Save 4(
5+) - Dmg:1HP [R:+0.4] [SUCCESS]
ADVANCE --- [01:11:54] E1 T1 P1 SHOOT : Unit 3(2,11) ADVANCED from (1,12) to (2,11) [Roll: 2] [R:+0.0] [SUCCESS]
CHARGE  --- [01:11:54] E1 T2 P2 CHARGE : Unit 101(1,4) CHARGED Unit 1(0,5) from (3,0) to (1,4) [Roll: 8] [R:+0.1] [SUCCESS]
FIGHT   --- [01:11:54] E1 T2 P2 FIGHT : Unit 101(1,4) FOUGHT Unit 1(0,5) with [Rending Claws] - Hit 3(2+) - Wound 1(4+) [R:+0.0] [FIGHT_SUBPHASE:ch
arging] [CHARGING_POOL:101] [ACTIVE_ALT_POOL:] [NON_ACTIVE_ALT_POOL:1] [SUCCESS]
Sample missing (0/5) : none

--------------------------------------------------------------------------------
2.7 CORE ISSUES
--------------------------------------------------------------------------------
Parsing errors (Non-standard log format): 0
Unit ID mismatches (Critical Bug):        0
```

Analyzer.py's Summary

```
================================================================================
SUMMARY
================================================================================
--------------------------------------------------------------------------------
PHASES
--------------------------------------------------------------------------------
✅ 1.1 Erreurs en phase de move : 0
✅ 1.2 Erreurs en phase de shooting : 0
✅ 1.3 Erreurs en phase de charge : 0
✅ 1.4 Erreurs en phase de fight : 0
✅ 1.5 Actions occuring in the wrong phase : 0
✅ 1.6 Double-activation par phase : 0
✅ 1.7 Special rules usage : 1380 utilisations
✅ 1.8 Weapon rules usage : 366 utilisations
--------------------------------------------------------------------------------
INTEGRITY
--------------------------------------------------------------------------------
✅ 2.1 Dead units interactions : 0
✅ 2.2 Positions/logs incohérents : 0
✅ 2.3 DMG issues : 0
✅ 2.4 Episodes duration : Min: 0.36s (E286) - Avg: 0.77s - Max: 1.54s (E253)
✅ 2.4 Episodes actions : Min: 63 (E25) - Avg: 147.3 - Max: 300 (E152)
✅ 2.5 Episode ending : 0
✅ 2.6 Sample missing (0/5) : none
✅ 2.7 Core issue : 0

################################################################################

✅ Aucune erreur détectée   -   Output : analyzer.log
```